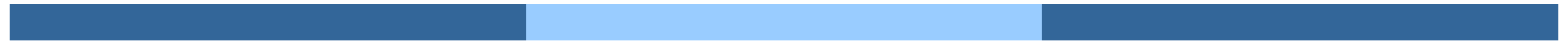


Virtual Memory



Computer System III
Wenbo Shen

Review

- Logical vs physical address
- Memory allocation
 - Contiguous allocation: first-, best-, worst-fit
 - Fragmentation: external vs internal
- Paging: page number + page offset
 - Hierarchical page table, hashed page table, inverted page table
 - 1-level vs 2-level, why save memory, page table walk
- Swapping
- MMU
 - TLB

Outline

- Demand paging
- Copy-on-write
- Page replacement algorithm
 - FIFO, optimal, LRU, ...
- Allocation of frames
- Thrashing
- Linux memory management

Background

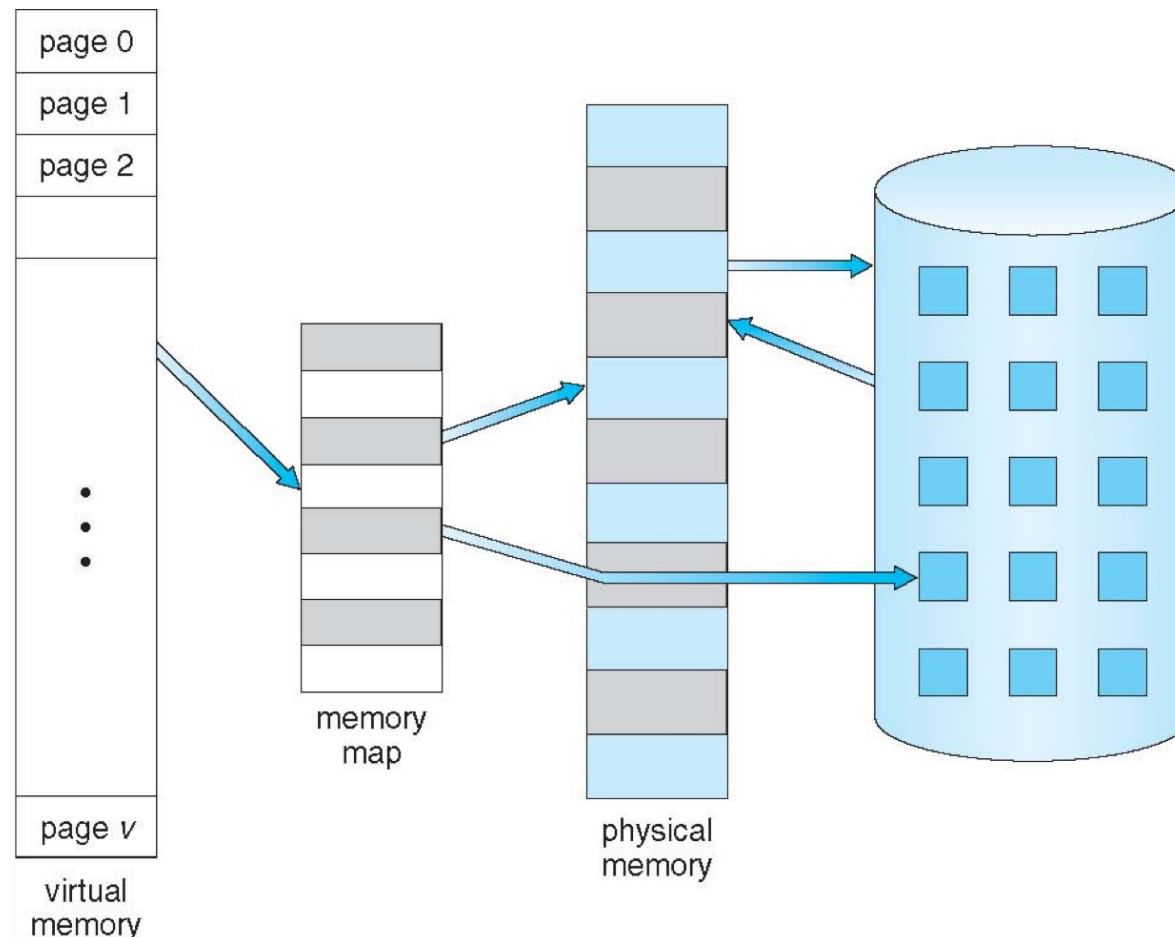
- Code needs to be in memory to execute, but entire program **rarely** needed or used at the same time
 - **Unused code**: error handling code, unusual routines
 - **Unused data**: large data structures
- Consider ability to execute **partially-loaded program**
 - Program no longer constrained by limits of physical memory
 - Programs could be larger than physical memory

Background

- Virtual memory: separation of **logical memory** from **physical memory**
 - Only part of the program needs to be in memory for execution
 - Logical address space can be much larger than physical address space
 - More programs can run concurrently
 - Less I/O needed to load or swap processes (part of it)
 - Allows memory (e.g., Shared library) to be shared by several processes
 - Better IPC performance
 - Allows for more efficient process **forking** (**copy-on-write**)
- Virtual memory can be implemented via:
 - **Paging**

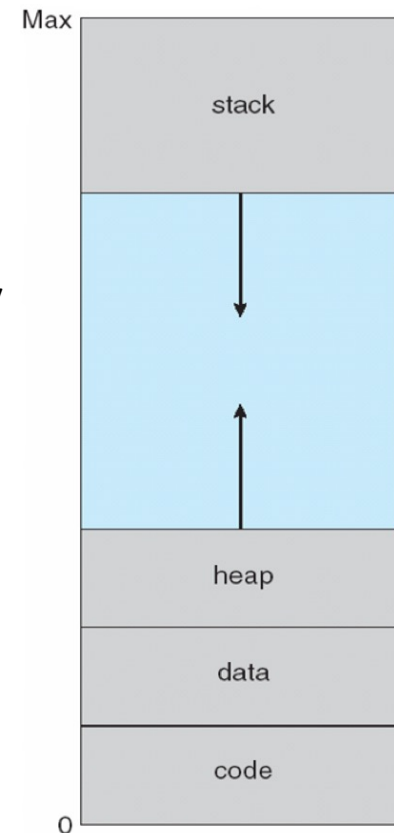
Virtual Memory Larger Than Physical Memory

- Virtual memory is larger than physical memory
 - Virtual memory is just address ranges, all storage is backed by physical memory

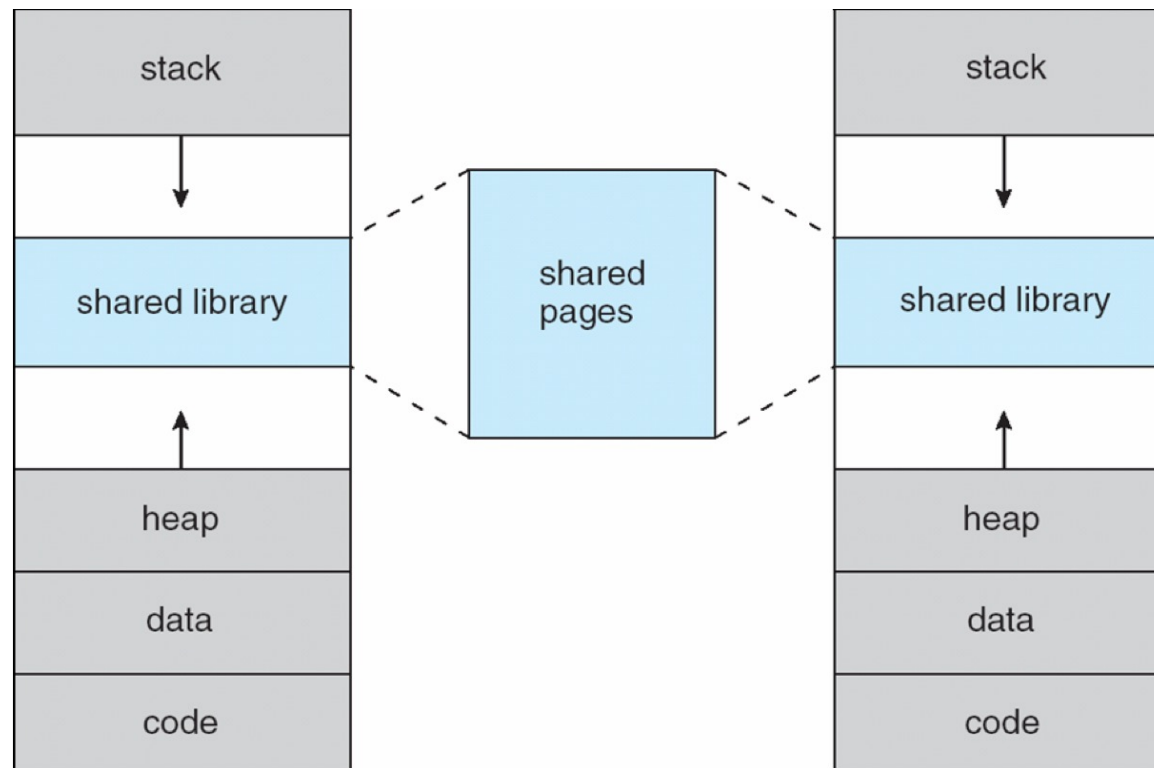


Virtual-address Space

- Usually design virtual address space for stack to start at Max logical address and grow “down” while heap grows “up”
 - Maximizes address space use
 - Unused address space between the two is hole
 - No physical memory needed until heap or stack grows to a given new page
- Enables **sparse** address spaces with holes left for growth, dynamically linked libraries, etc
- System libraries shared via mapping into virtual address space
- Pages can be shared during fork(), speeding process creation: COW



Shared Library Using Virtual Memory



Shared Library Using Virtual Memory

```
wenbo@parallels: ~  
wenbo@parallels: ~ 107x30  
7ffc75a5f000-7ffc75a80000 rw-p 00000000 00:00 0 [stack]  
7ffc75aa7000-7ffc75aaa000 r--p 00000000 00:00 0 [vvar]  
7ffc75aaa000-7ffc75aac000 r-xp 00000000 00:00 0 [vdso]  
ffffffffffff600000-ffffffffffff601000 r-xp 00000000 00:00 0 [vsyscall]  
wenbo@parallels:~$ which cat  
/bin/cat  
wenbo@parallels:~$ file /bin/cat  
/bin/cat: ELF 64-bit LSB shared object, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/l,  
for GNU/Linux 3.2.0, BuildID[sha1]=747e524bc20d33ce25ed4aea108e3025e5c3b78f, stripped  
wenbo@parallels:~$ cat /proc/self/maps  
55b793b79000-55b793b81000 r-xp 00000000 08:01 1048601 /bin/cat  
55b793d80000-55b793d81000 r--p 00007000 08:01 1048601 /bin/cat  
55b793d81000-55b793d82000 rw-p 00008000 08:01 1048601 /bin/cat  
55b794d33000-55b794d54000 rw-p 00000000 00:00 0 [heap]  
7f1974b90000-7f197555f000 r--p 00000000 08:01 662494 /usr/lib/locale/locale-archive  
7f197555f000-7f1975746000 r-xp 00000000 08:01 267596 /lib/x86_64-linux-gnu/libc-2.27.so  
7f1975746000-7f1975946000 ---p 001e7000 08:01 267596 /lib/x86_64-linux-gnu/libc-2.27.so  
7f1975946000-7f197594a000 r--p 001e7000 08:01 267596 /lib/x86_64-linux-gnu/libc-2.27.so  
7f197594a000-7f197594c000 rw-p 001eb000 08:01 267596 /lib/x86_64-linux-gnu/libc-2.27.so  
7f197594c000-7f1975950000 rw-p 00000000 00:00 0  
7f1975950000-7f1975977000 r-xp 00000000 08:01 267568 /lib/x86_64-linux-gnu/ld-2.27.so  
7f1975b3c000-7f1975b60000 rw-p 00000000 00:00 0  
7f1975b77000-7f1975b78000 r--p 00027000 08:01 267568 /lib/x86_64-linux-gnu/ld-2.27.so  
7f1975b78000-7f1975b79000 rw-p 00028000 08:01 267568 /lib/x86_64-linux-gnu/ld-2.27.so  
7f1975b79000-7f1975b7a000 rw-p 00000000 00:00 0  
7ffc73010000-7ffc73031000 rw-p 00000000 00:00 0 [stack]  
7ffc73148000-7ffc7314b000 r--p 00000000 00:00 0 [vvar]  
7ffc7314b000-7ffc7314d000 r-xp 00000000 00:00 0 [vdso]  
ffffffffffff600000-ffffffffffff601000 r-xp 00000000 00:00 0 [vsyscall]  
wenbo@parallels:~$
```

Demand Paging

- **Demand paging** brings a **page** into memory only when it is **demanded**
 - Demand means access (read/write)
 - If page is invalid (error) $\Rightarrow$ abort the operation
 - If page is valid but not in memory $\Rightarrow$ bring it to memory
 - Memory here means **physical** memory
 - This is called **page fault**
 - Via swapping for swapped pages
 - Via mapping for new page
 - No unnecessary I/O, less memory needed, slower response, more apps

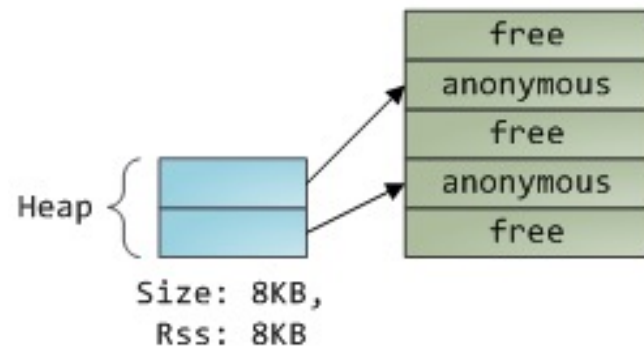
Demand Paging

- Demand paging vs page fault
 - What is the relationship?
- What causes page fault?
 - User space program accesses an address
- Which hardware issues page fault?
 - MMU
- Who handles page fault?
 - Operating system

What causes page fault?

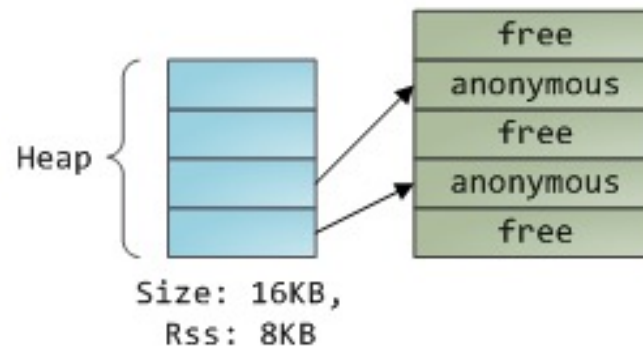
- User space program accesses an address

1. Program calls `brk()` to grow its heap

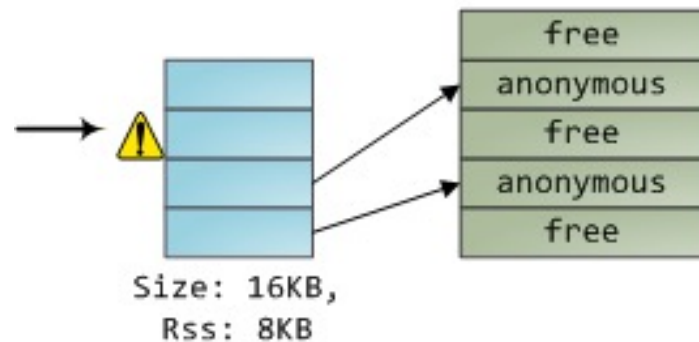


2. `brk()` enlarges heap VMA.

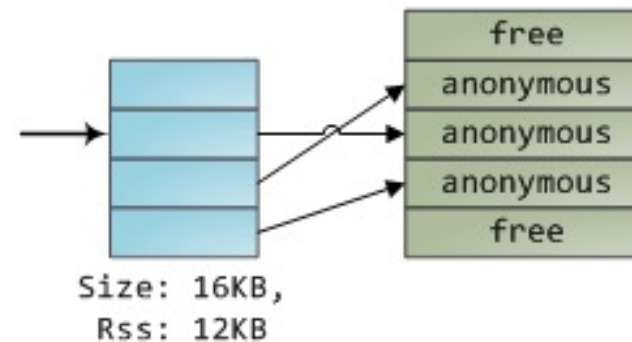
New pages are **not** mapped onto physical memory.



3. Program tries to access new memory.
Processor page faults.



4. Kernel assigns page frame to process,
creates PTE, resumes execution. Program is
unaware anything happened.



MMU issues page fault

- How does MMU know the physical frame is not mapped?
- Each page table entry has a valid–invalid (present) bit
 - $\underline{v}$ $\Rightarrow$ frame mapped, $\underline{i}$ $\Rightarrow$ frame not mapped
 - Initially, valid–invalid bit is set to $\underline{i}$ on all entries
 - During address translation, if the entry is invalid, it will trigger a **page fault**
- Example of a page table snapshot:

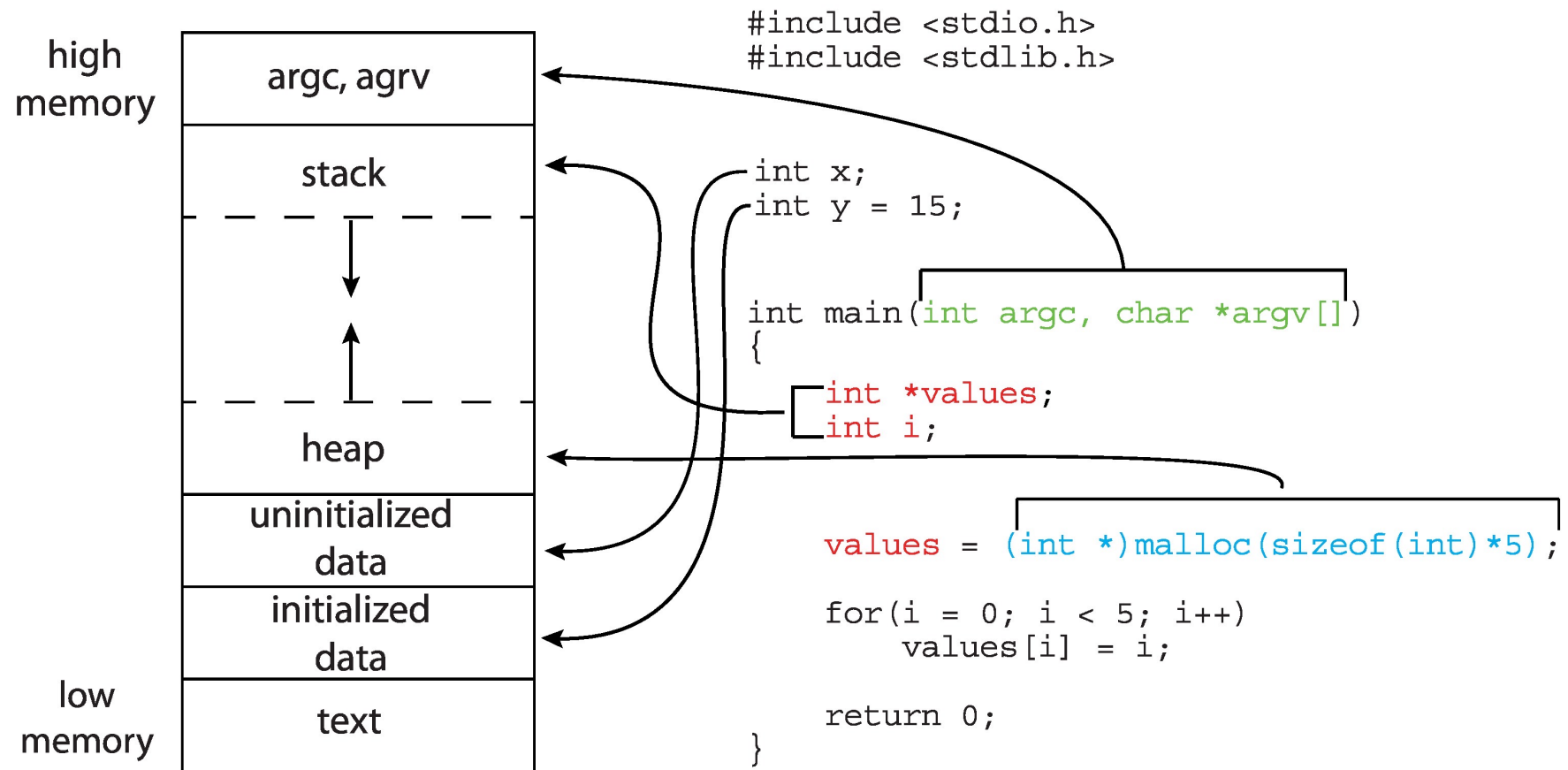
Frame #	v/i bit
	v
	v
	v
	v
	i
....	
	i
	i

page table

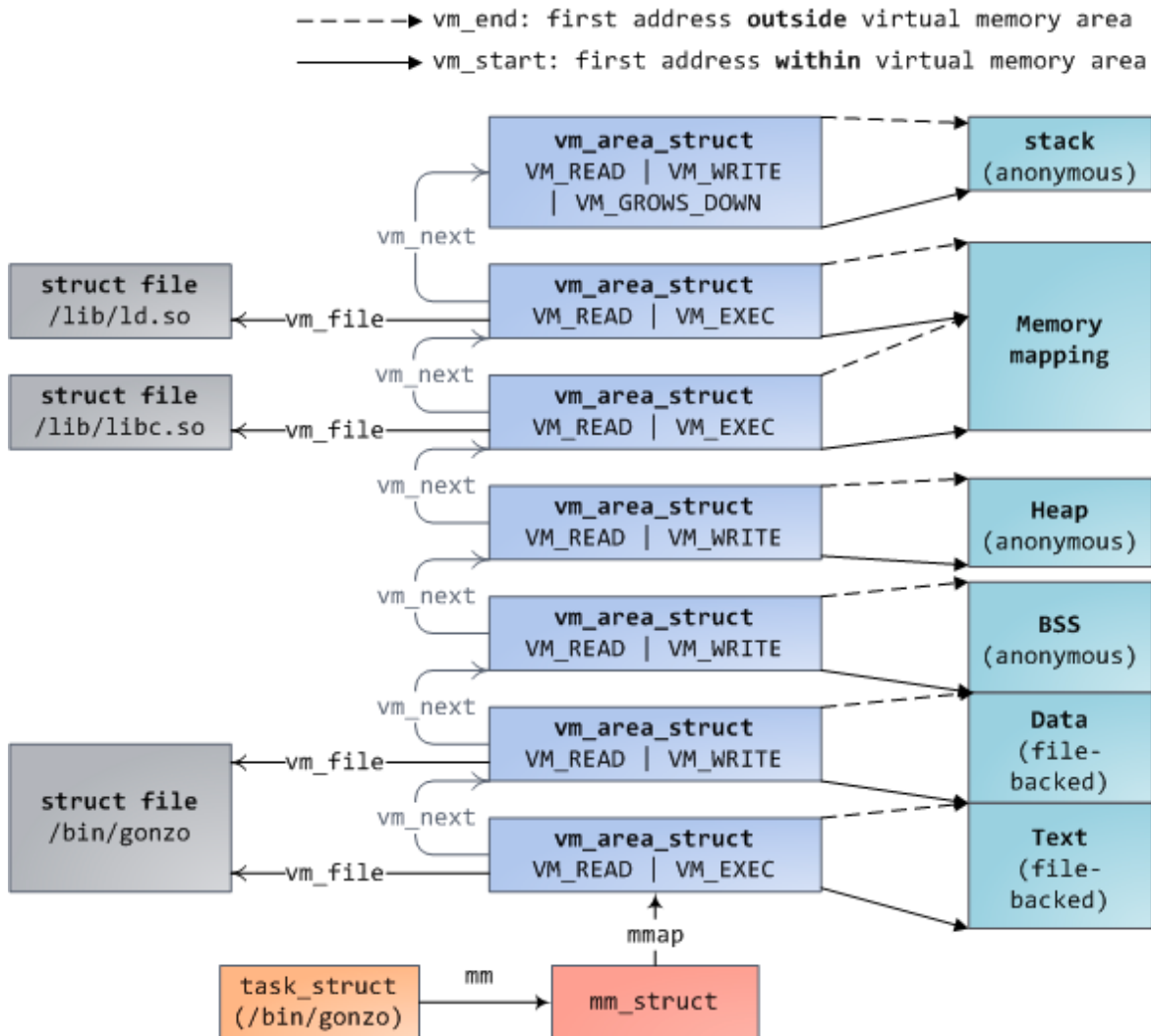
Who handles page fault?

- Linux implementation
 - First check vma to decide fault type
 - Address in vm_area
 - Address out of vm_area
 - Error, abort
 - Then map physical frame

Memory Layout of a C Program



Page Faults



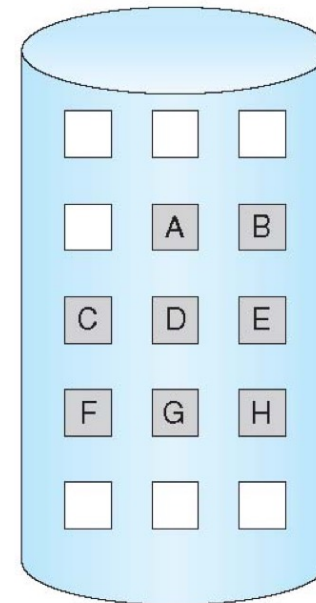
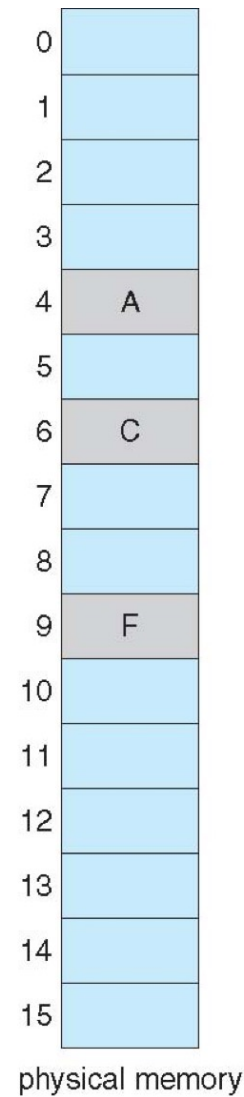
```

305 struct vm_area_struct {
306     /* The first cache line has the info for VMA tree walking. */
307
308     unsigned long vm_start;          /* Our start address within vm_mm. */
309     unsigned long vm_end;           /* The first byte after our end address
310                                     within vm mm. */

```

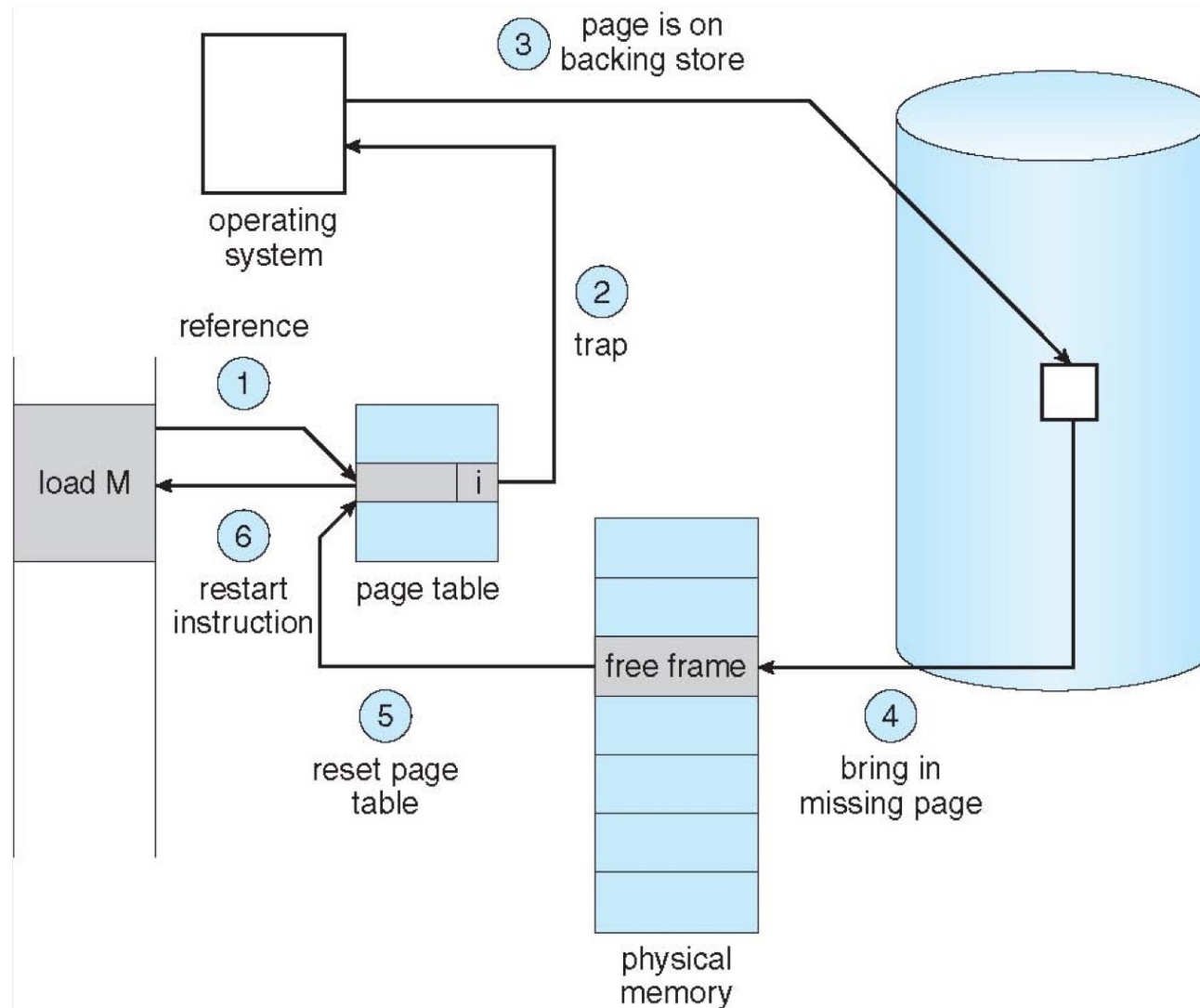

Page Table (Mem pages are not all in memory)

- Executable file



Page Fault Handling

- Page fault for code



Page Fault

- First reference to a non-present page will trap to kernel: **page fault**
- Operating system looks at memory mapping to decide:
 - **Invalid reference** ➡ deliver an exception to the process
 - Via check vma in linux
 - **Valid but not in physical memory** ➡ bring in
 - Get an empty physical frame
 - Bring page into frame via disk operation
 - Set page table entry to indicate the page is now in memory
 - Restart the instruction that caused the page fault

Page Fault - swapper

- **Lazy swapper**: never **swaps a page in** memory unless it will be needed
 - The swapper that deals with pages is also called a pager
- **Pre-paging**: pre-page all or some of pages a process will need, before they are referenced
 - It can reduce the number of page faults during execution
 - If pre-paged pages **are unused**, I/O and memory was wasted
 - Although it reduces page faults, total I/O# likely is higher

Page Fault

- Extreme case: start process with no frames in memory (aka. **Pure demand paging**)
 - OS sets instruction pointer to first instruction of process
 - Invalid page $\Rightarrow$ page fault
 - Every page is paged in on first access
 - **Program locality** reduces the overhead
 - An instruction could access multiple pages $\Rightarrow$ multiple page faults
 - e.g., Instruction, data, and page table entries for them
- Demand paging needs hardware support
 - Page table entries with **valid / invalid bit**
 - **Backing storage** (usually disks)
 - **Instruction restart**

Page Fault – Get Free Frame

- When a page fault occurs, the operating system must bring the desired page from secondary storage into main memory.
- Most operating systems maintain a free-frame list -- a pool of free frames for satisfying such requests.



- Operating system typically allocate free frames using a technique known as **zero-fill-on-demand** -- the content of the frames zeroed-out before being allocated.
- When a system starts up, all available memory is placed on the free-frame list.

Stages in Demand Paging – Worse Case

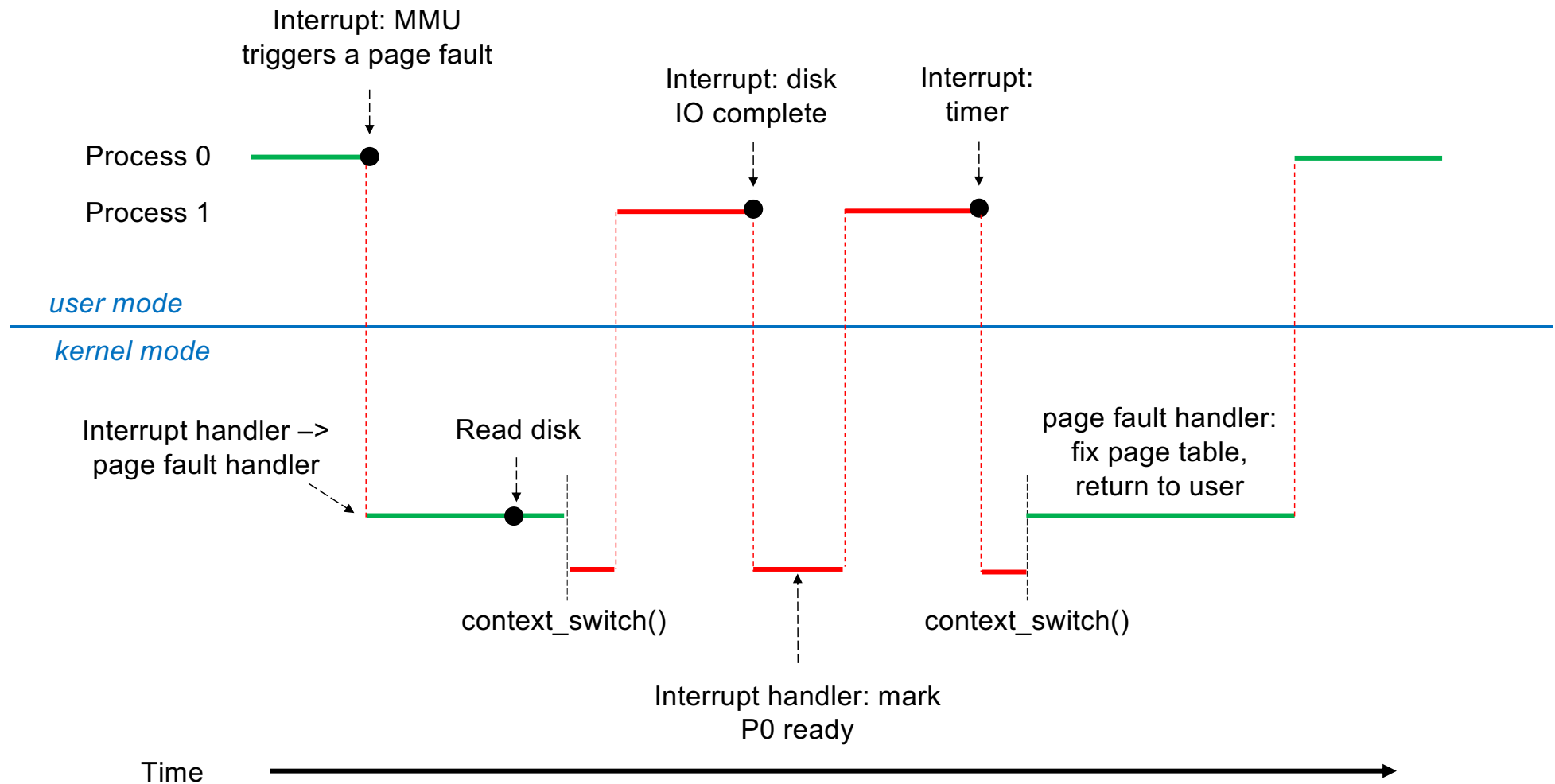
- 1. Trap to the operating system
- 2. Save the user registers and process state
- 3. Determine that the interrupt was a page fault
 - 3.1 Check that the page reference was legal
- 4. Find a free frame
- 5. Determine the location of the page on the disk, issue a read from the disk to the free frame:
 - 5.1 Wait in a queue for this device until the read request is serviced
 - 5.2 Wait for the device seek and/or latency time
 - 5.3 Begin the transfer of the page to a free frame

Stages in Demand Paging – Worse Case

- 6. While waiting, allocate the CPU to other process
- 7. Receive an interrupt from the disk I/O subsystem (I/O completed)
 - 7.1 Determine that the interrupt was from the disk
 - 7.2 Mark page fault process ready
- 8. Wait for the CPU to be allocated to this process again
 - 8.1 Save registers and process state for other process
 - 8.2 Context switch to page fault process
- 9. Correct the page table, mapping new frame
- 10. Return to user: restore the user registers, process state, and new page table, and then resume the interrupted instruction

Stages in Demand Paging – Worse Case

- Assume a page fault happen in Process 0
- While waiting for disk, the CPU is allocated to Process 1



Demand Paging: EAT

- Page fault rate: $0 \leq p \leq 1$
 - if $p = 0$ no page faults
 - if $p = 1$, every reference is a fault
- Effective Access Time (EAT):
 $(1 - p) \times \text{memory access} + p \times (\text{page fault overhead} + \text{swap page out} + \text{swap page in} + \text{instruction restart overhead})$

Demand Paging Example

- Assume memory access time: 200 nanoseconds, average page-fault service time: 8 milliseconds
 - $EAT = (1 - p) \times 200 + p \times (8 \text{ milliseconds})$
 $= (1 - p) \times 200 + p \times 8,000,000$
 $= 200 + p \times 7,999,800$
 - If one out of 1,000 causes a page fault, then $EAT = 8.2$ microseconds
 - A slowdown by a factor of 41000
 - $8.2 \text{ ms} / 0.2 \text{ us} = 41000$
 - If want < 10 percent, less than one page fault in every 400,000 accesses

Demand Paging Optimizations

- Swap space I/O faster than file system I/O even if on the same device
 - Swap allocated in larger chunks, less management needed than file system
- Copy entire process image from **disk** to **swap space** at process load time
 - Then page in and out of swap space
 - Used in older BSD Unix

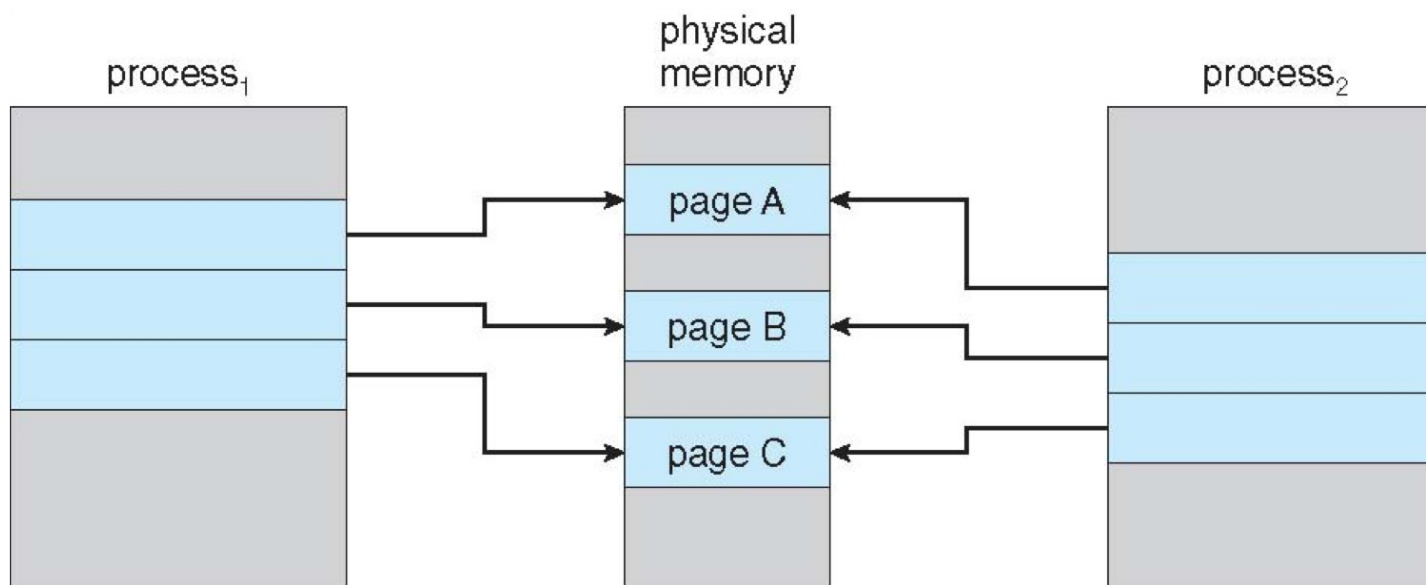
Demand Paging Optimizations

- Demand page in from program binary on disk, but **discard** rather than paging out when freeing frame (and reload from disk next time)
 - Following cases still need to **write** to swap space
 - Pages not associated with a file (like stack and heap) – anonymous memory
 - Pages modified in memory but not yet written back to the file system
- Mobile systems
 - Typically don't support swapping
 - Instead, demand page from **file system** and reclaim read-only pages (such as code)

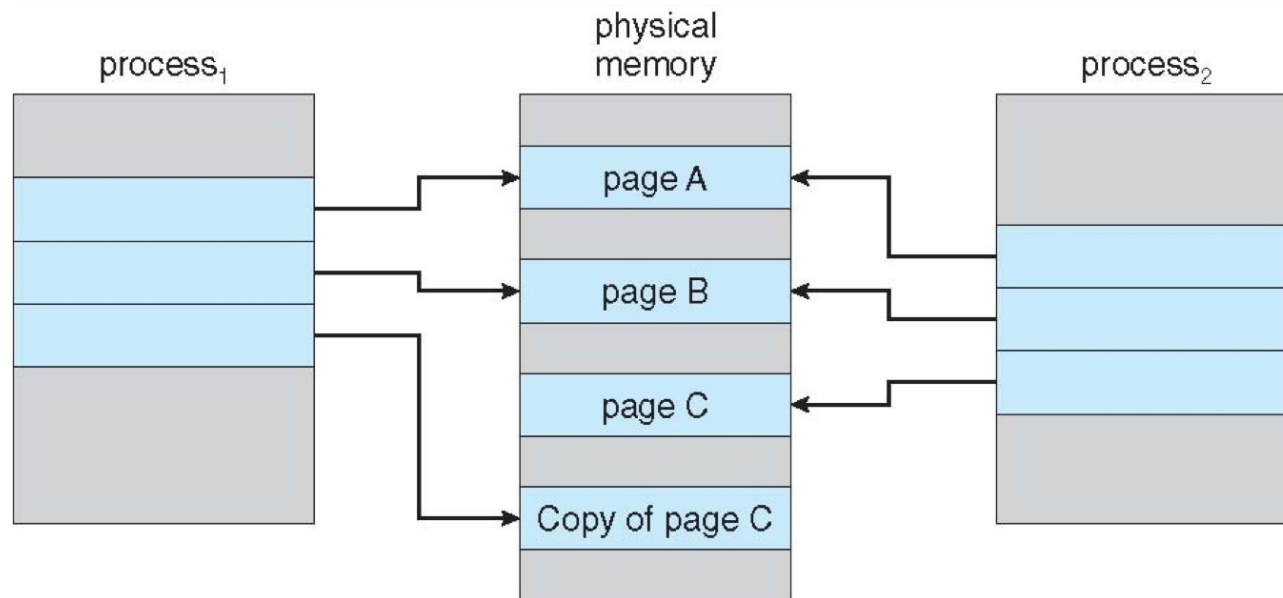
Copy-on-Write

- **Copy-on-write** (COW) allows parent and child processes to initially share the same pages in memory
 - The page is shared as long as no process modifies it
 - If either process modifies a shared page, only then is the page copied
- COW allows more efficient **process creation**
 - No need to copy the parent memory during fork
 - Only changed memory will be copied later
- vfork syscall optimizes the case that child calls **exec** immediately after fork
 - Parent is suspended until child exits or calls exec
 - Child shares the parent resource, including the heap and the stack
 - Child cannot return from the function or call exit, should call `_exit`
 - vfork could be fragile, **it is invented when COW has not been implemented**

Before Process 1 Modifies Page C



After Process 1 Modifies Page C



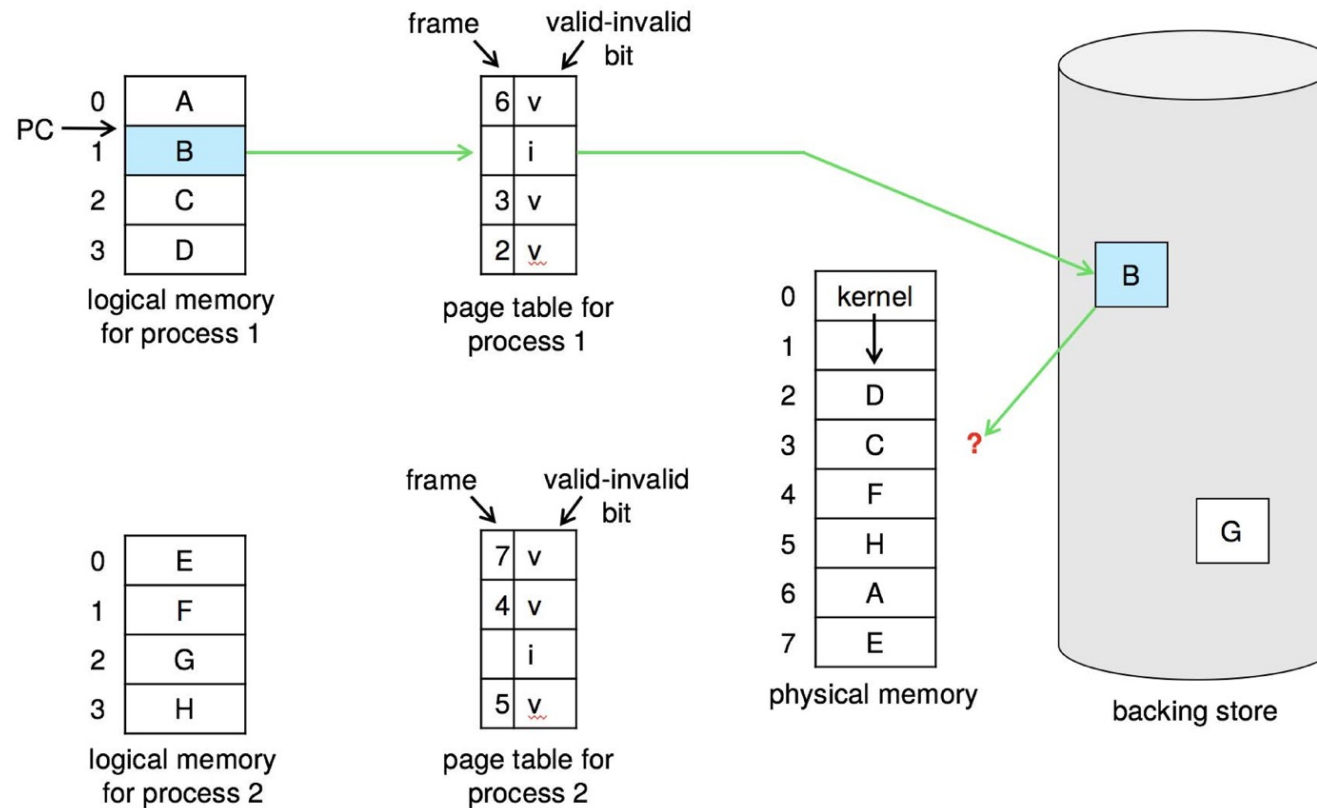
What Happens if There is no Free Frame?

- Used up by process pages
- Also in demand from the kernel, I/O buffers, etc
- How much to allocate to each?
- **Page replacement** – find some page in memory, but not really in use, page it out
 - Algorithm – terminate? swap out? replace the page?
 - Performance – want an algorithm which will result in minimum number of page faults
- Same page may be brought into memory several times

Page Replacement

- Memory is an important resource, system may run out of memory
- To prevent out-of-memory, swap out some pages
 - Page replacement usually is a part of the page fault handler
 - Policies to select victim page require careful design
 - Need to reduce overhead and avoid **thrashing**
 - Use modified (dirty) bit to reduce number of pages to swap out
 - Only modified pages are written to disk
 - Select some processes to kill (last resort)
- **Page replacement completes separation between logical memory and physical memory** – large virtual memory can be provided on a smaller physical memory

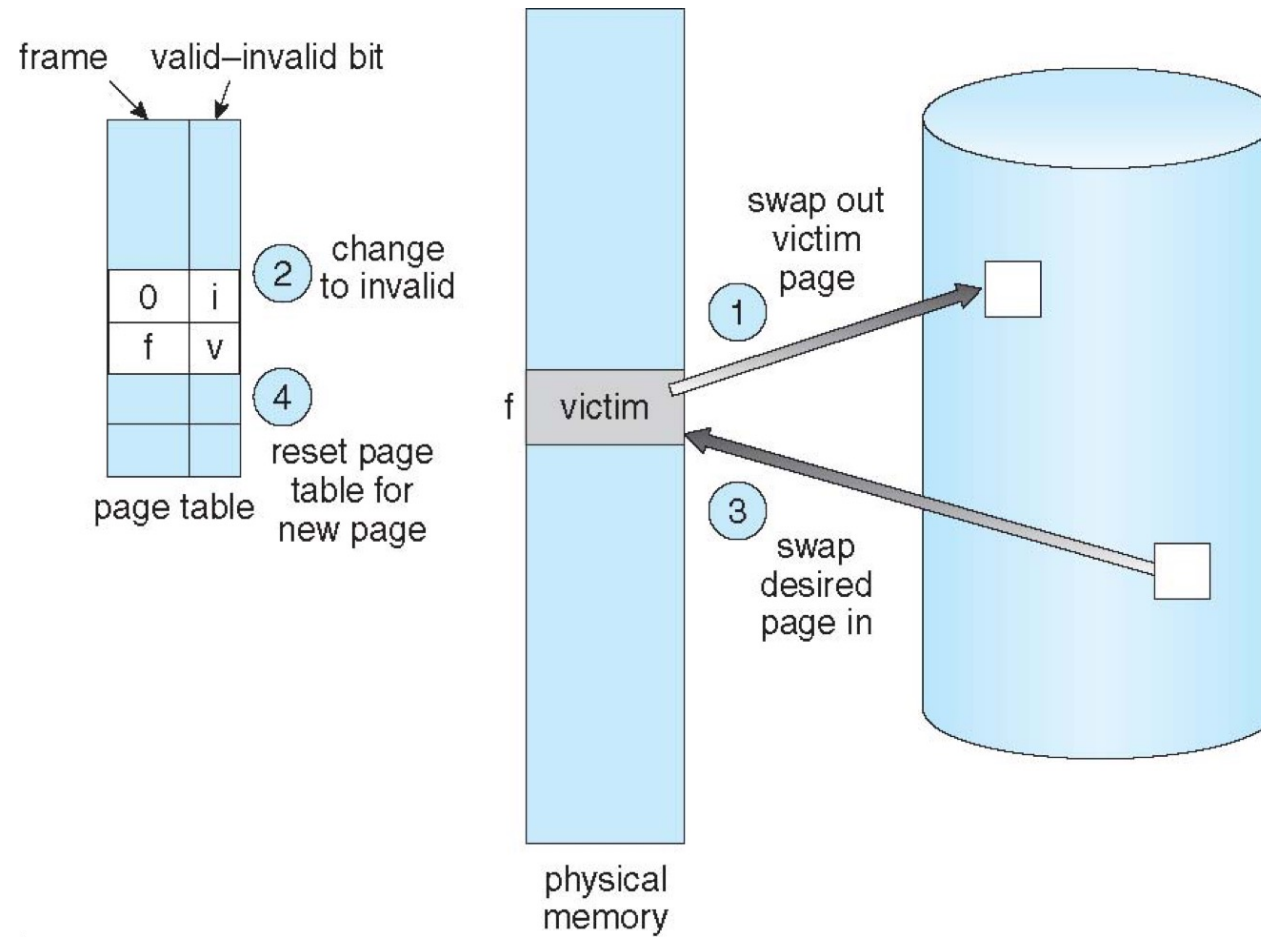
Need For Page Replacement



Page Fault Handler (with Page Replacement)

- To **page in** a page:
 - Find the location of the desired page on disk
 - Find a free frame:
 - If there is a free frame, use it
 - If there is none, use a page replacement policy to pick a victim frame, write victim frame to disk if dirty
 - Bring the desired page into the free frame; update the page tables
 - Restart the instruction that caused the trap
- Note now potentially **2 page I/O** for **one page fault** ➡ increase EAT

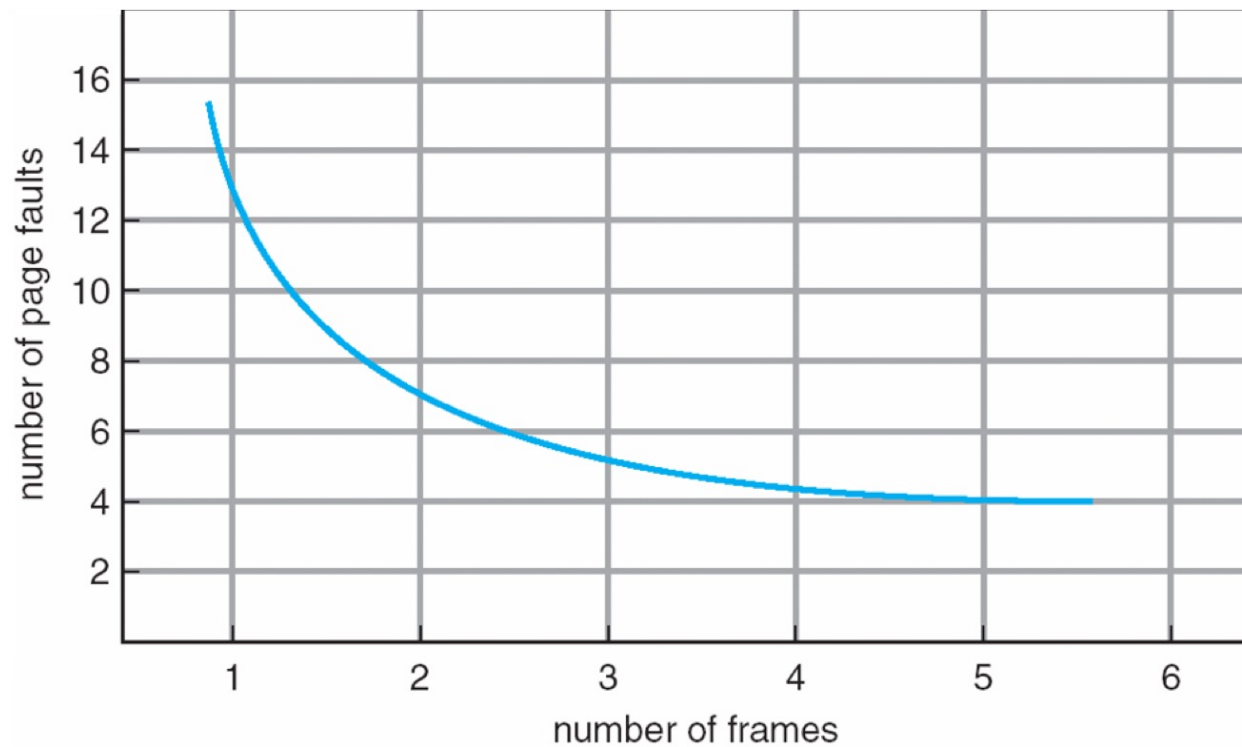
Page Replacement



Page Replacement Algorithms

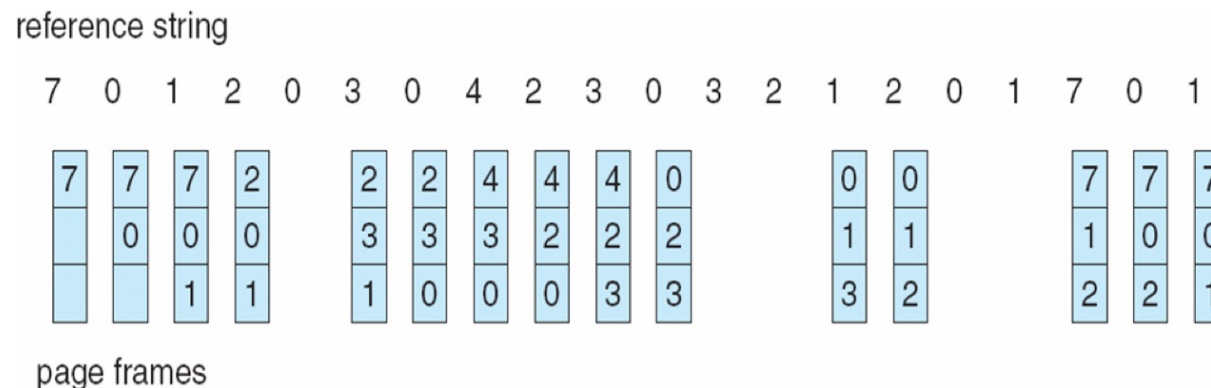
- Page-replacement algorithm should have lowest page-fault rate on both first access and re-access
 - FIFO, optimal, LRU, LFU, MFU...
- To evaluate a page replacement algorithm:
 - Run it on a particular string of memory references (reference string)
 - String is just page numbers, not full addresses
 - Compute the number of page faults on that string
 - Repeated access to the same page does not cause a page fault
 - In all our examples, the reference string is
7,0,1,2,0,3,0,4,2,3,0,3,0,3,2,1,2,0,1,7,0,1

Page Faults v.s. Number of Frames



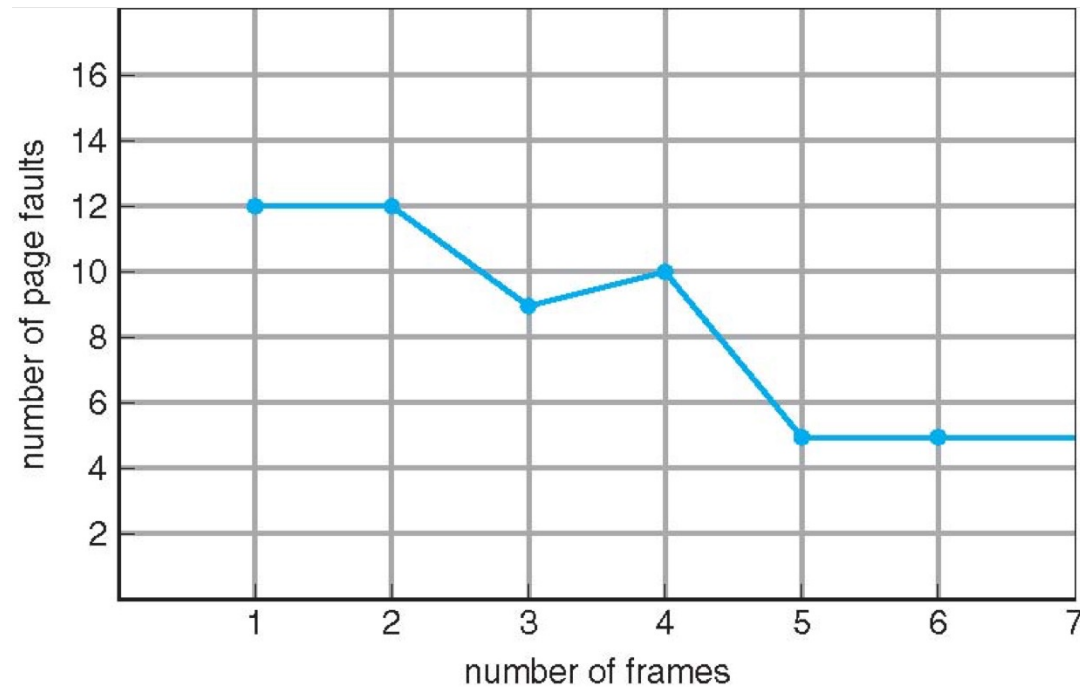
First-In-First-Out (FIFO)

- **FIFO**: replace the first page loaded
 - Similar to sliding a window of n in the reference string
 - Our reference string 7,0,1,2,0,3,0,4,2,3,0,3,0,3,2,1,2,0,1,7,0,1
 - Will cause 15 page faults with 3 frames
 - How about reference string of 1,2,3,4,1,2,5,1,2,3,4,5 /w 3 or 4 frames?
- For FIFO, adding **more frames** can cause **more page faults**!
- **Belady's Anomaly**



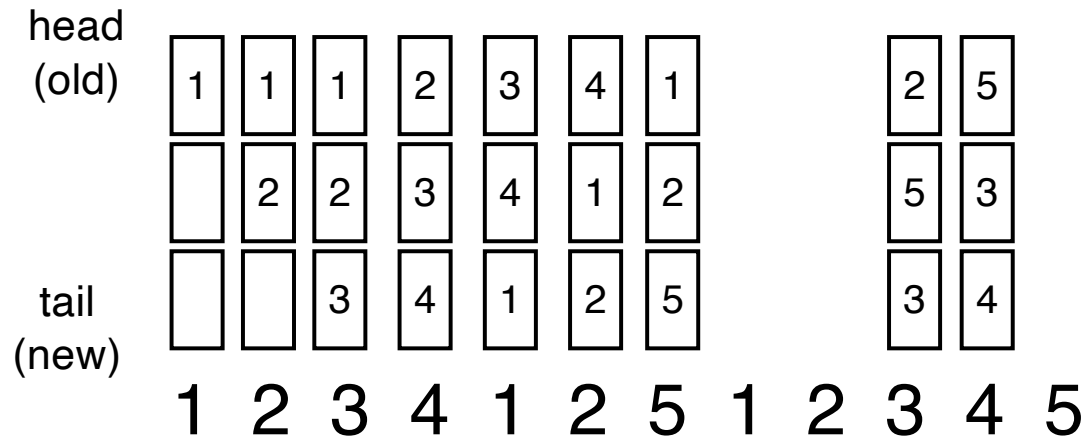
15 page faults

FIFO Illustrating Belady's Anomaly

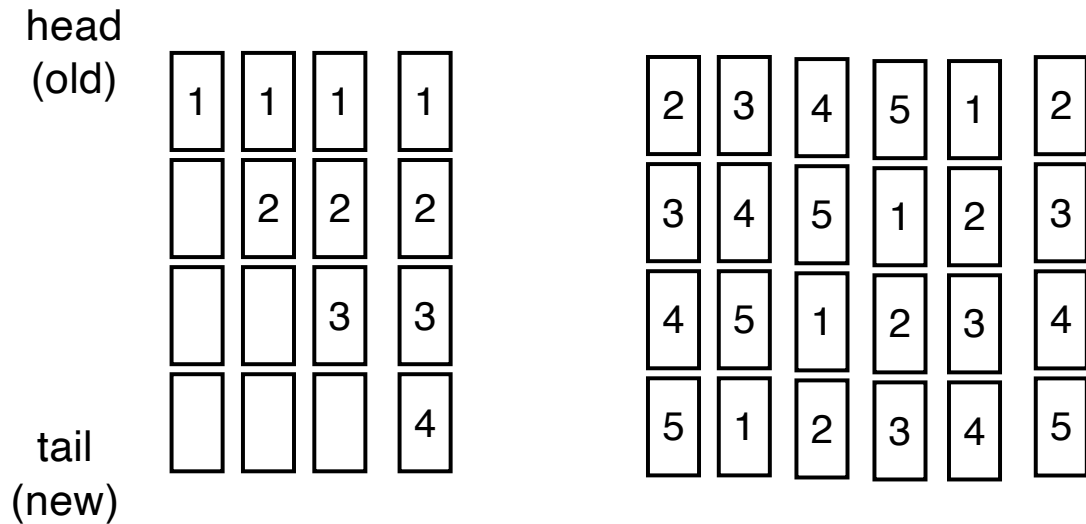


1 2 3 4 1 2 5 1 2 3 4 5

Belady's Anomaly



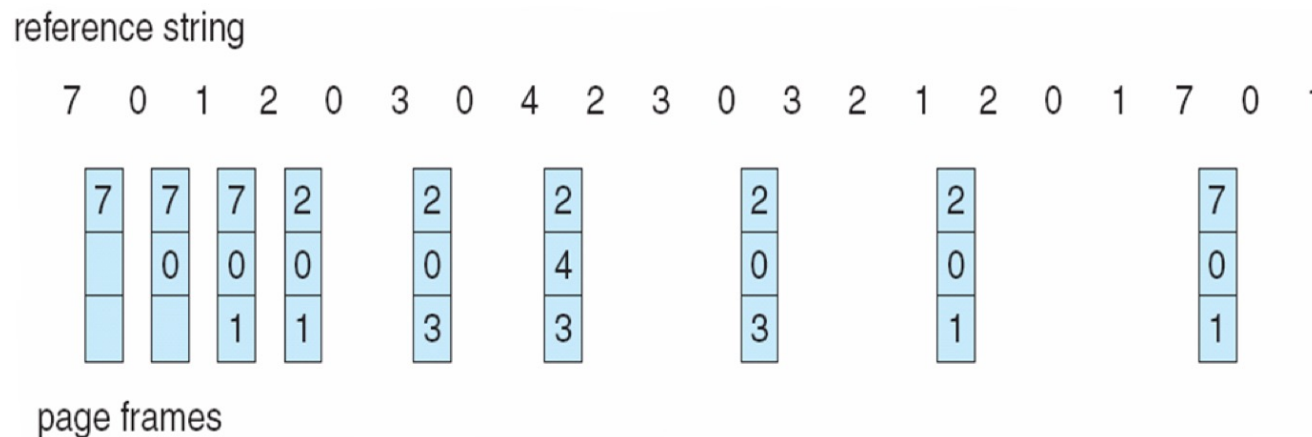
9 page faults



10 page faults!

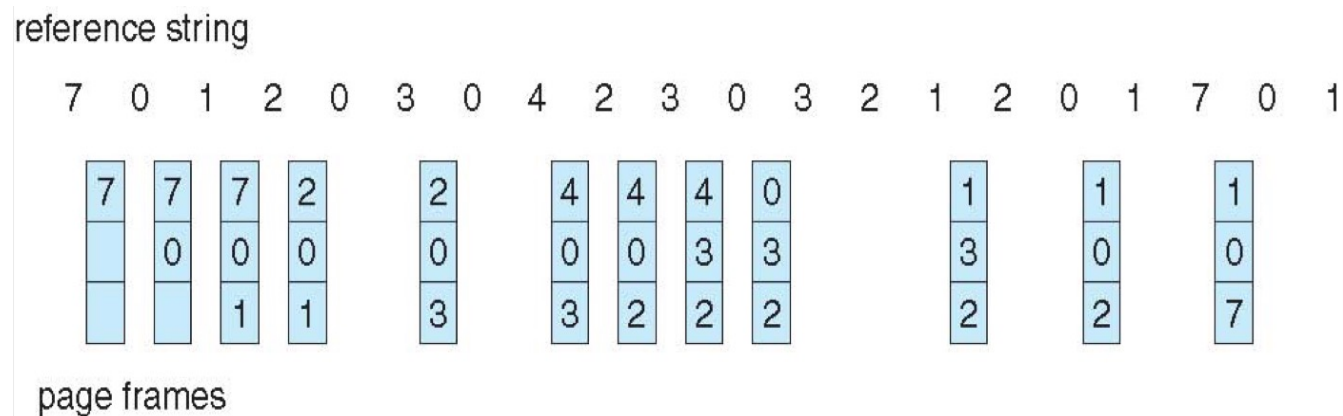
Optimal Algorithm

- **Optimal** : replace page that will not be used for the longest time
 - 9-page-fault is optimal for the example on previous slide
- How do you know which page will not be used for the longest time?
 - Can't read the future
 - Used for measuring how well your algorithm performs



Least Recently Used (LRU)

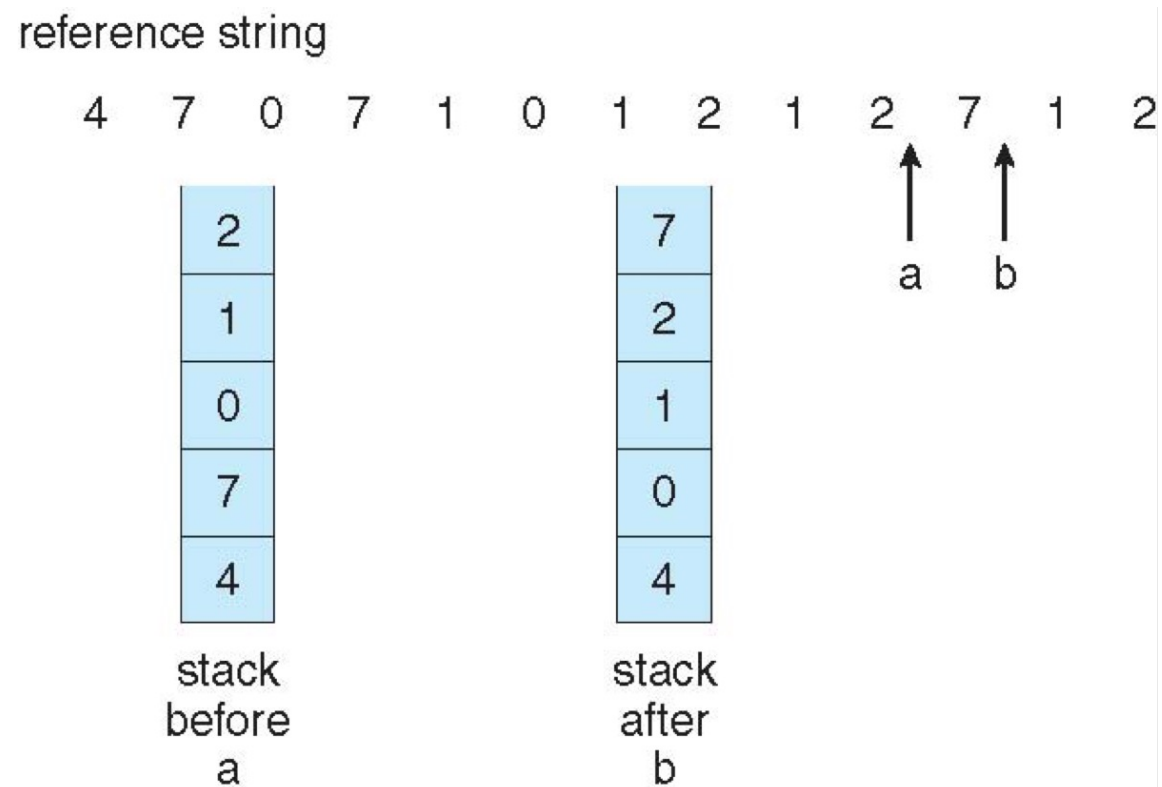
- **LRU** replaces pages that have not been used for the longest time
 - Associate time of last use with each page, select pages w/ oldest timestamp
 - Generally good algorithm and frequently used
 - 12 faults for our example, better than FIFO but worse than OPT
- LRU and OPT do **NOT** have Belady's anomaly
- How to implement LRU?
 - **Counter-based**
 - **Stack-based**



LRU Implementation

- **Counter-based** implementation
 - Every page table entry has a counter
 - Every time page is referenced, copy the **clock** into the counter
 - When a page needs to be replaced, search for page with smallest counter
 - Min-heap can be used
- **Stack-based** implementation
 - Keep a stack of page numbers (in double linked list)
 - When a page is referenced, move it to the top of the stack
 - Each update is more expensive, but no need to search for replacement

Stack-based LRU



LRU Approximation Implementation

- Counter-based and stack-based LRU have high performance overhead
- Hardware provides **a reference bit**
- LRU approximation with a **reference bit**
 - Associate with each page a reference bit, initially set to 0
 - When page is referenced, set the bit to 1 (done by the hardware)
 - Replace any page with reference bit = 0 (if one exists)
 - We do not know the **order**, however

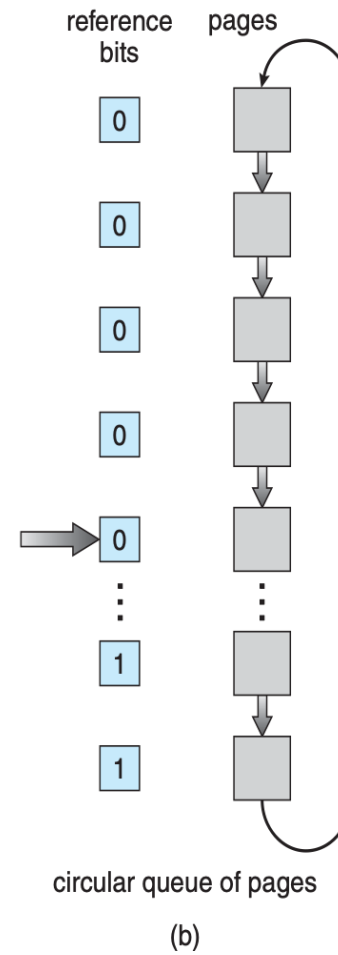
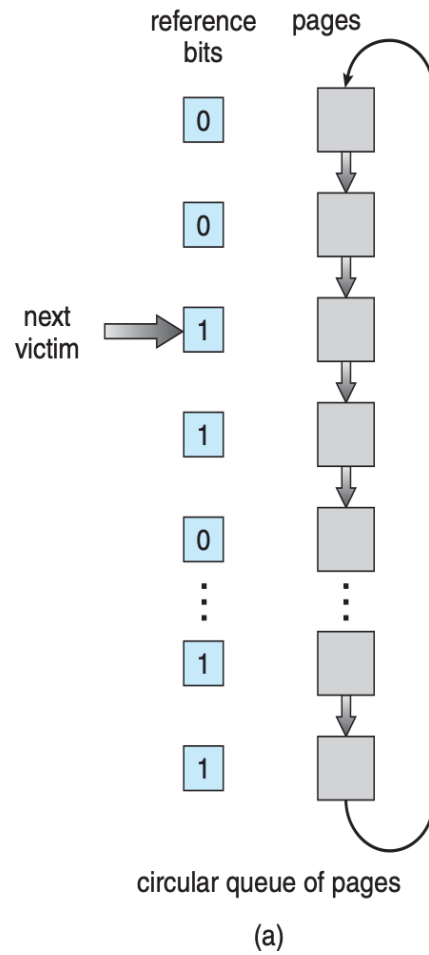
Additional-Reference-Bits Algorithm

- Reordering the bits at regular intervals
 - Suppose we have 8-bits byte for each page
 - During a time interval (100ms), shifts bit rights by 1 bit, sets the high bit if used, and then discards the low-order bits
 - 00000000 => has not been used in 8 time intervals
 - 11111111 => has been used in all time intervals
 - 11000100 vs 01110111 : which one is used more recently?

LRU Implementation

- **Second-chance** algorithm
 - Generally FIFO, plus hardware-provided **reference bit**
 - **Clock** replacement
 - If page to be replaced has
 - Reference bit = 0 -> replace it
 - Reference bit = 1 then:
 - Set reference bit 0, leave page in memory
 - Replace next page, subject to same rules

Second-chance (clock) Page-replacement Algorithm



Enhanced Second-Chance Algorithm

- Improve algorithm by using **reference bit** and modify bit (if available) in concert
- Take ordered pair (reference, modify):
 - (0, 0) neither recently used nor modified – best page to replace
 - (0, 1) not recently used but modified – not quite as good, must write out before replacement
 - (1, 0) recently used but clean – probably will be used again soon
 - (1, 1) recently used and modified – probably will be used again soon and need to write out before replacement
- When page replacement called for, use the clock scheme but use the four classes replace page in lowest non-empty class
 - Might need to search circular queue several times

Counting-based Page Replacement

- Keep the number of references made to each page
- **Least frequently used (LFU)** replaces page with the smallest counter
 - A page is heavily used during process initialization and then never used
- **Most frequently used (MFU)** replaces page with the largest counter
 - Based on the argument that page with the smallest count was probably just brought in and has yet to be used
- LFU and MFU are not common

Page-Buffering Algorithms

- **Keep a pool of free frames, always**
 - **Frame available when needed**, no need to find at fault time
 - Read page into free frames without waiting for victims to write out
 - Restart as soon as possible
 - When convenient, evict victim
- Possibly, keep list of **modified pages**
 - When disk **idles**, write pages there and set to non-dirty: this page can be replaced **without** writing pages to backing store
- Possibly, keep free frame contents intact and note what is in them - a kind of cache
 - **If referenced again before reused, no need to load contents again from disk**
 - **Cache hit**

Applications and Page Replacement

- All of these algorithms have OS guessing about future page access
- Some applications have better knowledge – i.e., databases
- Memory intensive applications can cause **double buffering - a waste of memory**
 - OS keeps copy of page in memory as I/O buffer
 - Application keeps page in memory for its own work
- Operating system can given direct access to the disk, getting out of the way of the applications
 - **Raw disk** mode
- Bypasses buffering, locking, etc

Allocation of Frames

- Each process needs **minimum number** of frames -according to instructions semantics
- Example: IBM 370 – **6 pages to handle SS MOVE** instruction:
 - Instruction is 6 bytes, might span 2 pages
 - 2 pages to handle from
 - 2 pages to handle to
- **Maximum** of course is total frames in the system
- Two major allocation schemes for **process** memory allocation
 - Equal allocation
 - Proportional allocation

Frame Allocation

- Equal allocation
 - For example, if there are 100 frames (after allocating frames for the OS) and 5 processes, give each process 20 frames
 - Keep some as free frame buffer pool
- Proportional allocation
 - Allocate according to the size of process
 - Dynamic as degree of multiprogramming, process sizes change

s_i = size of process p_i

$S = \sum s_i$

m = total number of frames

a_i = allocation for $p_i = \frac{s_i}{S} \times m$

$m = 62$

$s_1 = 10$

$s_2 = 127$

$a_1 = \frac{10}{137} \times 62 \approx 4$

$a_2 = \frac{127}{137} \times 62 \approx 57$

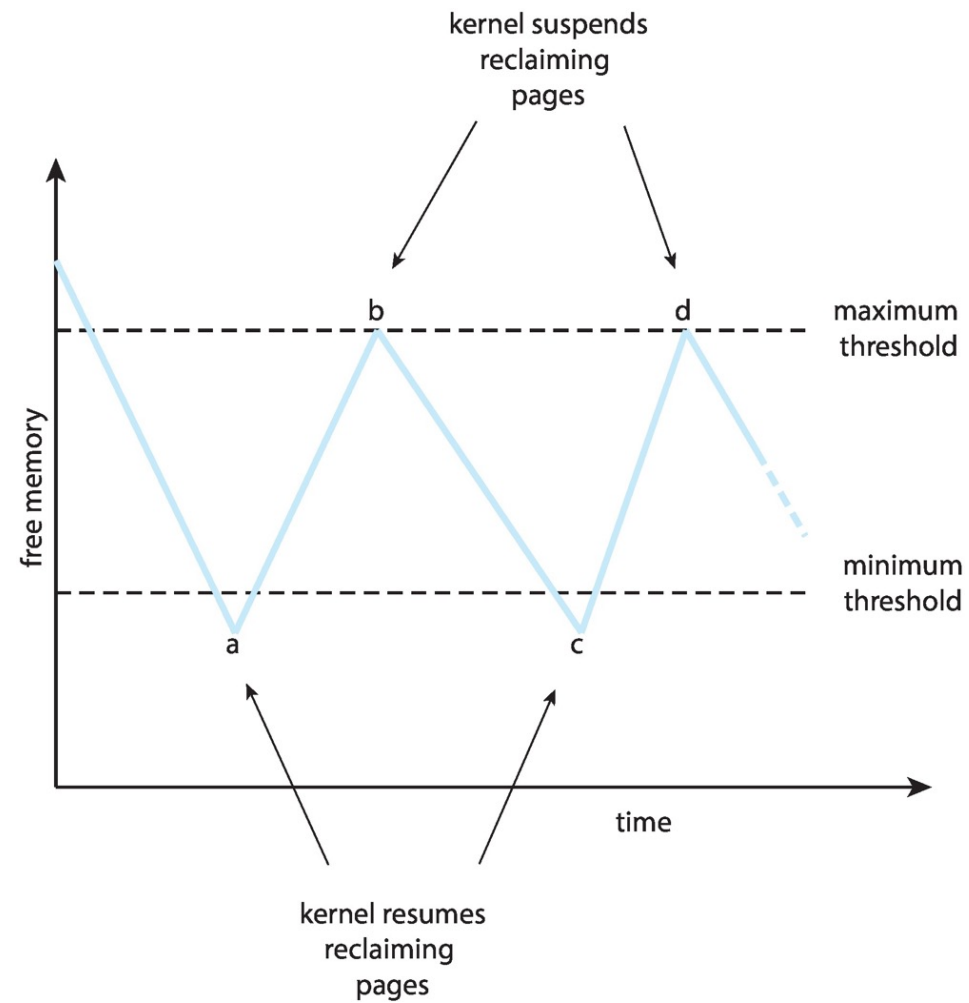
Global vs. Local Allocation

- **Global replacement** – process selects a replacement frame from the set of all frames; one process can take a frame from another
 - But then process execution time can **vary** greatly - depends on others
 - But greater throughput so more common
- **Local replacement** – each process selects from only its own set of allocated frames
 - More consistent per-process performance
 - But possibly underutilized memory

Reclaiming Pages

- A strategy to implement global page-replacement policy
- All memory requests are satisfied from the free-frame list
- Rather than waiting for the list to drop to zero before we begin selecting pages for replacement, page replacement is triggered when the list falls below a **certain threshold**.
- This strategy attempts to ensure **there is always sufficient** free memory to satisfy new requests.

Reclaiming Pages Example



What happens if memory is below the minimum threshold

- Reclaim pages aggressively
 - Kill some processes
 - According to OOM score
 - How likely it is to be terminated in case of low available memory

Major and minor page faults

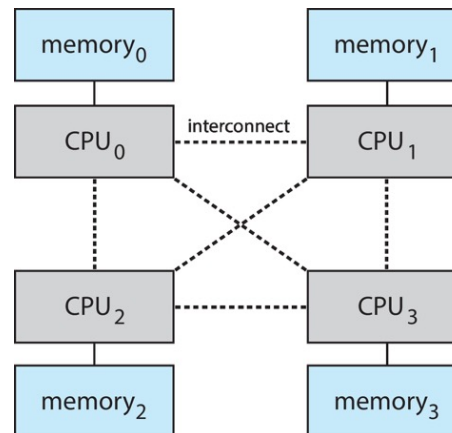
- Major: page is referenced but not in memory
 - Can only be satisfied by disk
 - `do_anonymous_page` is not major
- Minor: mapping does not exist, but the page is in memory
 - Shared library
 - Reclaimed and not freed yet

```
wenbo@parallels:~$ ps eo minflt,majflt,cmd
MINFL  MAJFL  CMD
    704      1 /usr/lib/gdm3/gdm-x-session --run-script env
 59284     17 /usr/lib/xorg/Xorg vt2 -displayfd 3 -auth /ru
 8439      12 /usr/lib/gnome-session/gnome-session-binary -
73769     25 /usr/bin/gnome-shell USER=wenbo LC_TIME=zh_CN
 1346       0 ibus-daemon --xim --panel disable USER=wenbo
   376       1 /usr/lib/ibus/ibus-dconf USER=wenbo LC_TIME=z
 1659       0 /usr/lib/ibus/ibus-x11 --kill-daemon USER=wen
```

Thanks to shared libraries!

Non-Uniform Memory Access

- So far all memory accessed equally
- Many systems are NUMA – speed of access to memory varies
 - Consider system boards containing CPUs and memory, interconnected over a system bus
- NUMA multiprocessing architecture



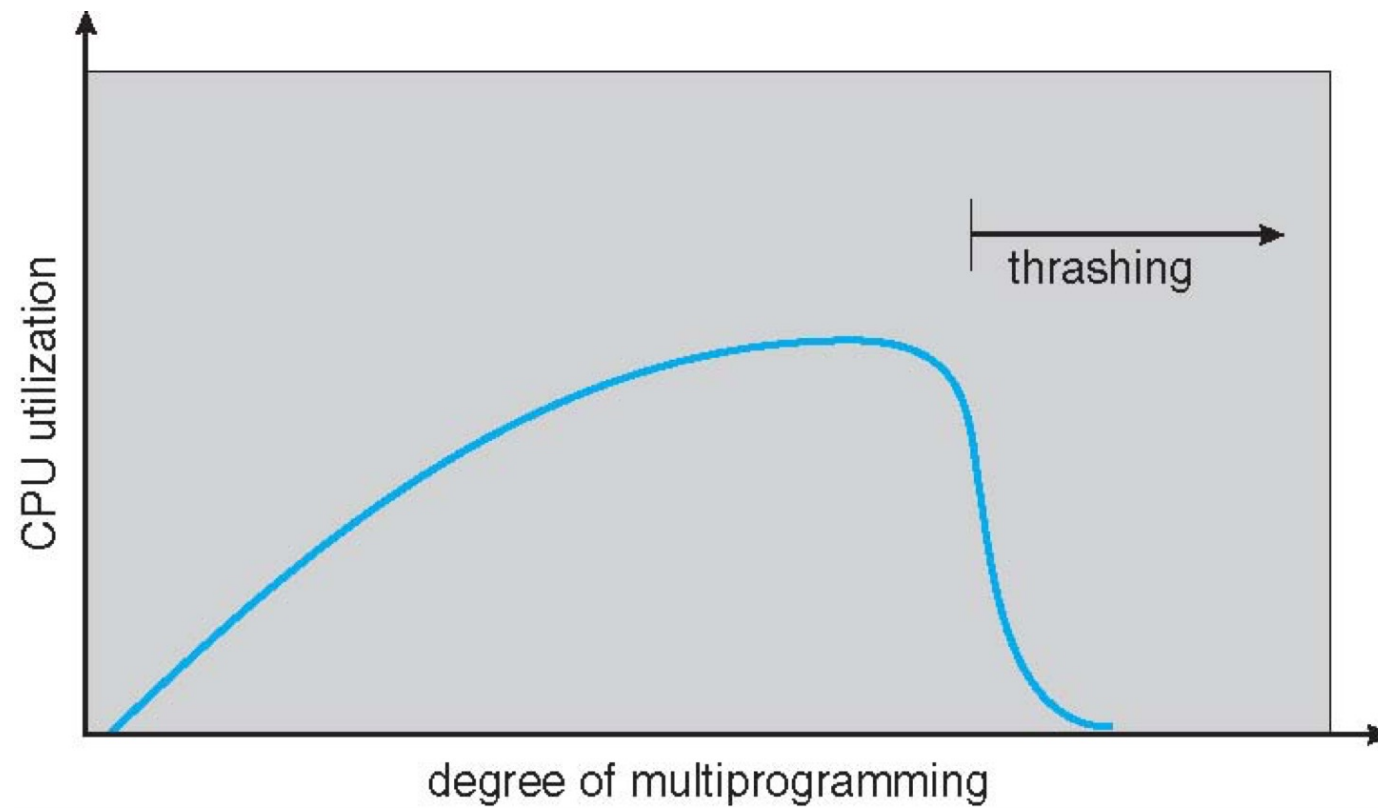
Non-Uniform Memory Access (Cont.)

- Optimal performance comes from allocating memory “close to” the CPU on which the thread is scheduled
 - And modifying the **scheduler** to schedule the thread on the same system board when possible
 - Linux
 - Kernel maintains scheduling domains: does not allow threads to migrate across domains
 - A separate free-frame list for each NUMA node - allocating memory from the node it is running

Thrashing

- If a process doesn't have "enough" pages (frames), page-fault rate may be high
 - Page fault to get page, replace some existing frame
 - But quickly need replaced frame back
 - This leads to: low CPU utilization $\Rightarrow$ kernel thinks it needs to increase the degree of multiprogramming to maximize CPU utilization $\Rightarrow$ more processes added to the system
- **Thrashing**: a process is busy swapping pages in and out

Thrashing



Demand Paging and Thrashing

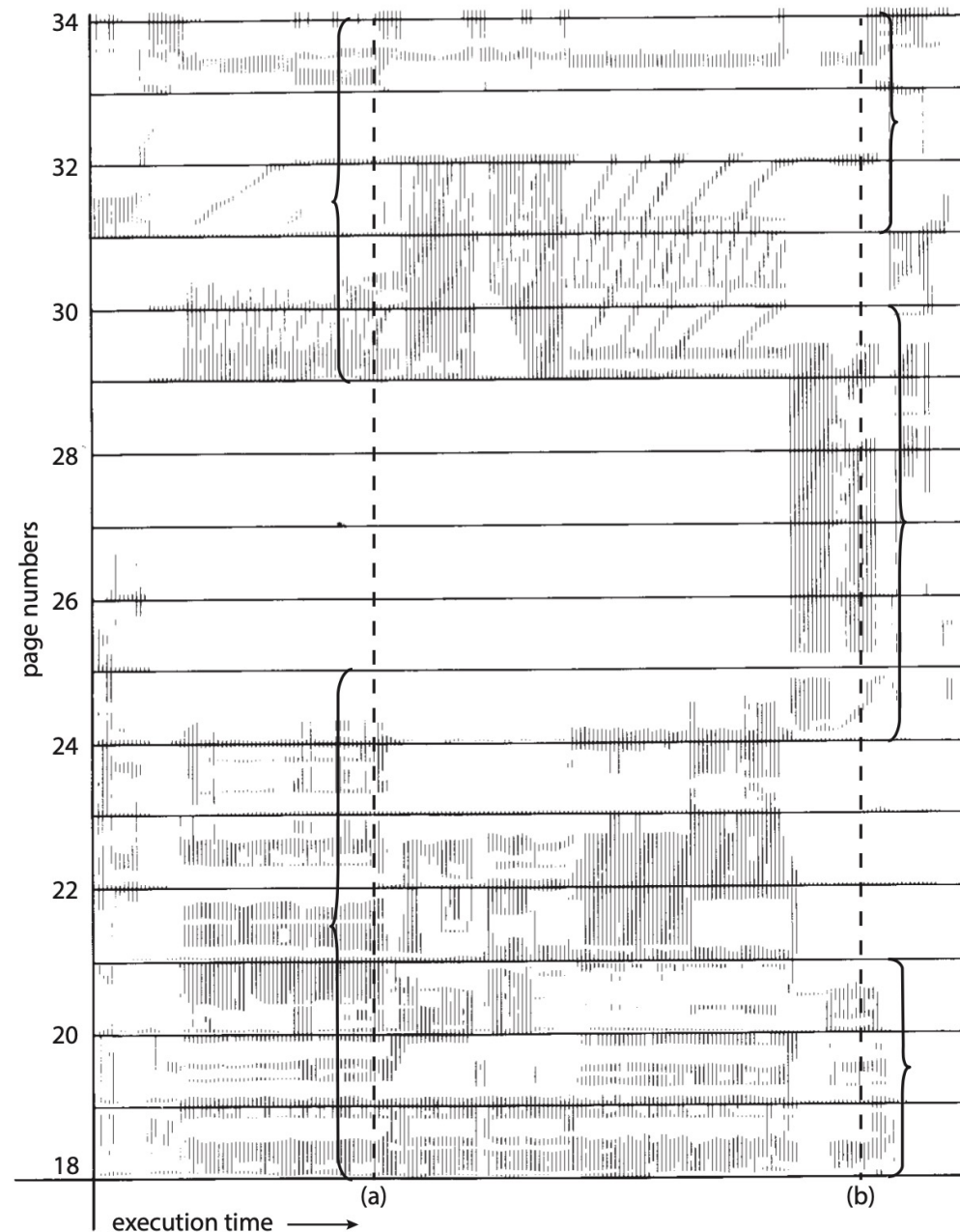
- Why does demand paging work?
 - Process memory access has **high locality**
 - Process migrates from one locality to another, localities may overlap
- Why does thrashing occur?
 - Total memory size < total size of locality

Resolve thrashing - Option I

- Limit thrashing effects by **using local page** replacement
 - One process starts thrashing does not affect others -> it cannot cause other processes thrashing
 - Select swap out page from the same process

Resolve thrashing - Option II

- Provide a process with as many frames as it needs. How?
 - Time (a), the locality is the set of pages {18-24, 29-33}.
 - Time (b), the locality changes to {18-20, 24-29, 31-33}
 - 18, 19, and 20 are part of both localities

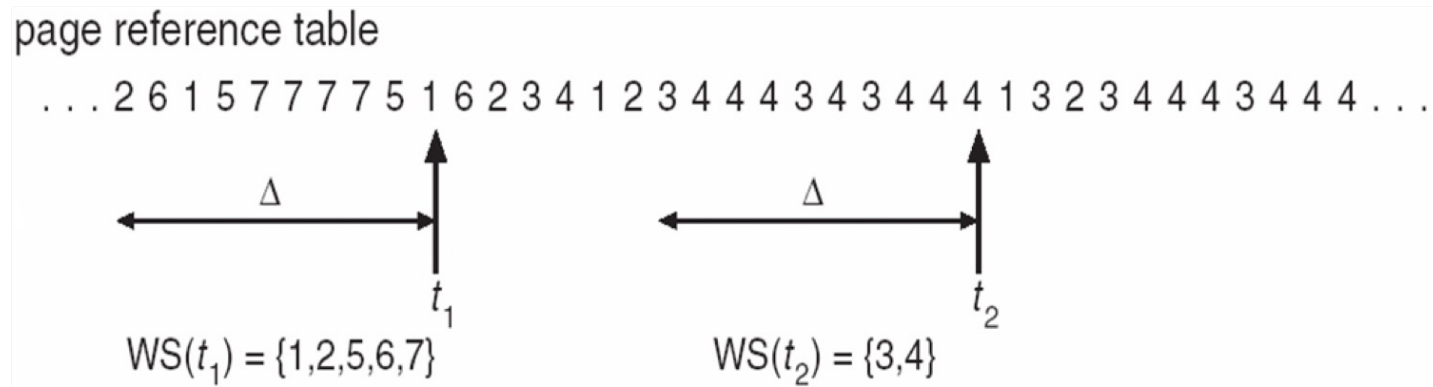


Working-Set Model

- **Working-set window(Δ):** a fixed number of page references
 - If Δ too small $\Rightarrow$ will not include entire locality
 - If Δ too large $\Rightarrow$ will include several localities
 - If $\Delta = \infty \Rightarrow$ will include entire program
- **Working set** of process p_i (WSS_i): total number of pages referenced in the most recent Δ (varies in time)
- **Total working sets:** $D = \sum WSS_i$
 - Approximation of total locality
 - If $D > m \Rightarrow$ possibility of thrashing
 - To avoid thrashing: if $D > m$, suspend or swap out some processes

Working-Set Model

- Working-set window $\Delta = 10$

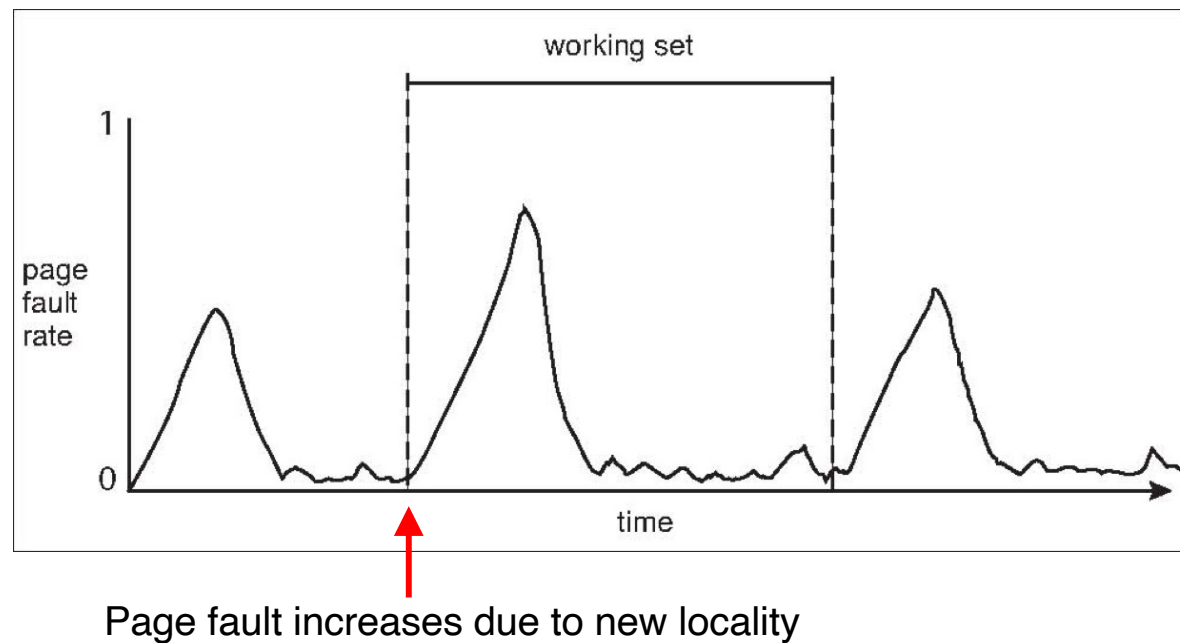


Challenge: Keeping Track of the Working Set

- Approximate with interval timer + a reference bit per page
 - The reference bit is set to 1 by hardware when accessing a page
- Example: $\Delta = 10,000$ time units
 - Timer interrupts after every 5000 time units
 - Keep 2 **memory bits** in memory for each page
 - Whenever a timer interrupts
 - OS copies the reference bit (to memory bits [0] or [1]) and sets the values of all reference bits to 0
 - If one of the **memory bits** = 1 $\Rightarrow$ page in working set
- Why is this not completely accurate?
 - 2 memory bits can only record two interrupts
 - Can not tell **when (in 5000 time units)** the access occurs
- Improvement = 10 bits and interrupt every **1000** time units

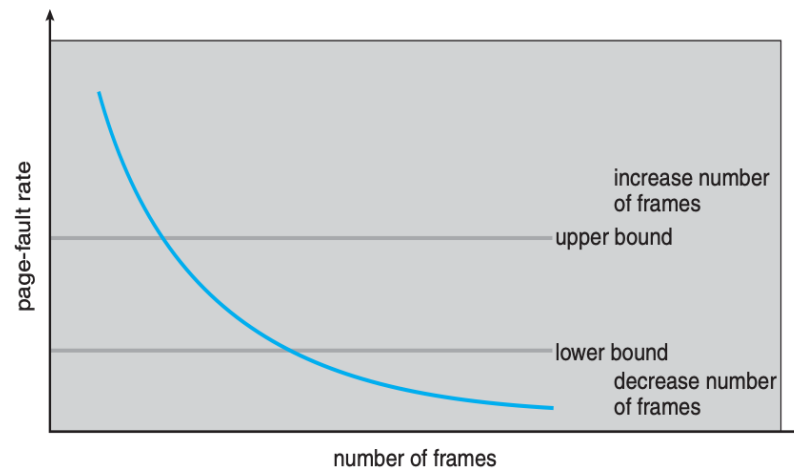
Working Sets and Page Fault Rates

- Assumes there is no thrashing
- Direct relationship between working set of a process and its page-fault rate
- Working set changes over time
- Peaks and valleys over time



Page-Fault Frequency

- More direct approach than WSS
- Establish “acceptable” page-fault frequency (PFF) rate
 - If actual rate too low, process loses frame
 - If actual rate too high, process gains frame
- Need to swap out a process if no free frames are available



Kernel Memory Allocation

- Kernel memory allocation is treated differently from user memory, it is often allocated from a free-memory pool
 - kernel requests memory for structures of varying sizes -> minimize waste due to fragmentation
 - Some kernel memory needs to be **physically contiguous**
 - e.g., for device I/O

Other Considerations

- Prepaging
- Page size
- TLB reach
- Program structure
- I/O interlock and page locking

Prepaging

- To reduce the large number of page faults that occurs at process startup
 - Prepage all or some of the pages a process will need, before they are referenced
 - But if prepaged pages are unused, I/O and memory was wasted
- Assume s pages are prepaged and a fraction a of these pages is used
 - Is cost of $s * a$ save pages faults $>$ or $<$ than the cost of prepaging
 - $s * (1 - a)$ unnecessary pages?
 - a near zero $\rightarrow$ prepaging loses

Page Size

- Sometimes OS designers have a choice
 - Especially if running on custom-built CPU
- Page size selection must take into consideration:
 - Fragmentation -> small page size
 - Page table size -> large page size
 - Resolution -> small page size
 - I/O overhead -> large page size
 - Number of page faults -> large page size
 - Locality -> small page size
 - TLB size and effectiveness -> large page size
- Always power of 2, usually in the range 2^{12} (4,096 bytes) to 2^{22} (4,194,304 bytes)
- On average, **growing over time**

TLB Reach

- **TLB reach:** the amount of memory accessible from the TLB
 - $\text{TLB reach} = (\text{TLB size}) \times (\text{page size})$
- Ideally, the working set of each process is stored in the TLB
 - Otherwise there is a high degree of TLB misses
- **Increase the page size** to reduce **TLB pressure**
 - It may increase fragmentation as not all applications require large page sizes
 - Multiple page sizes allow applications that require larger page sizes to use them without an increase in fragmentation

Other Issues: Program Structure

- Program structure can affect page faults
 - `int[128,128] data;`
 - Assume page size is 512B; each row is stored in one page;
 - Assume system has 127 physical frames
 - Program 1:

```
for (j = 0; j < 128; j++)  
    for (i = 0; i < 128; i++)  
        data[i,j] = 0;
```

128 x 128 = 16,384 page faults

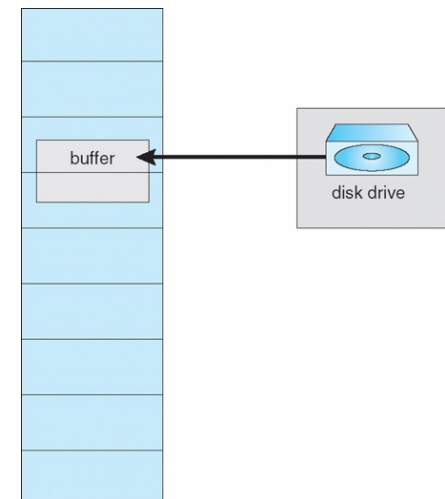
- Program 2:

```
for (i = 0; i < 128; i++)  
    for (j = 0; j < 128; j++)  
        data[i,j] = 0;
```

128 page faults

I/O interlock

- I/O Interlock – Pages must sometimes be locked into memory
- Consider I/O - Pages that are used for copying a file from a device must be **locked from being selected for eviction by a page replacement algorithm**



Windows XP

- Uses demand paging with clustering
 - Clustering brings in **pages surrounding the faulting page**: locality
- Processes are assigned **working set** minimum and set maximum
 - *wsmin*: minimum number of pages the process is guaranteed to have
 - *wsmax*: a process may be assigned as many pages up to its *wsmax*
- When the amount of free memory in the system falls below a threshold:
 - Automatic working set trimming to restore the amount of free memory
 - It removes pages from processes that have more pages than the *wsmin*

Memory management in Linux

- mm_struct
 - https://elixir.bootlin.com/linux/v6.0/source/include/linux/mm_types.h#L486
 - Where is page table?
 - Where is kernel page table?
 - Code walk through

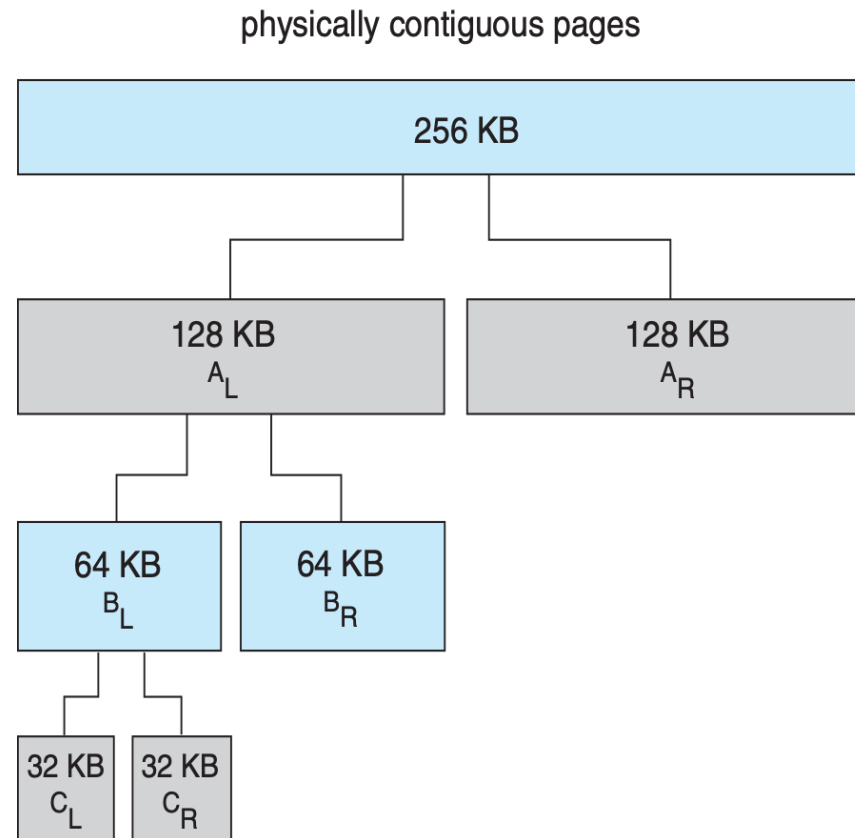
Page fault handling in Linux

- `do_page_fault`
 - <https://elixir.bootlin.com/linux/v6.0/source/arch/arm64/mm/fault.c#L519>
 - Code walk through
- `__handle_mm_fault`
 - <https://elixir.bootlin.com/linux/v6.0/source/mm/memory.c#L4967>
 - Page table walk

Linux Buddy System

- Memory allocated using power-of-2 allocator
 - Memory is allocated in units of the size of **power of 2**
 - Round up a request to the closest allocation unit
 - Split the unit into two “**buddies**” until a proper sized chunk is available
 - Assume only 256KB chunk is available, kernel requests 21KB
 - Split it into a_l and a_r of 128KB each
 - Further split an 128KB chunk into b_l and b_r of 64KB
 - Again, split a 64KB chunk into c_l and c_r of 32KB each
 - Give one chunk for the request
- Advantage: it can quickly merge unused chunks into larger chunk
- Disadvantage: **internal fragmentation**
 - 21k request -> 32k segment

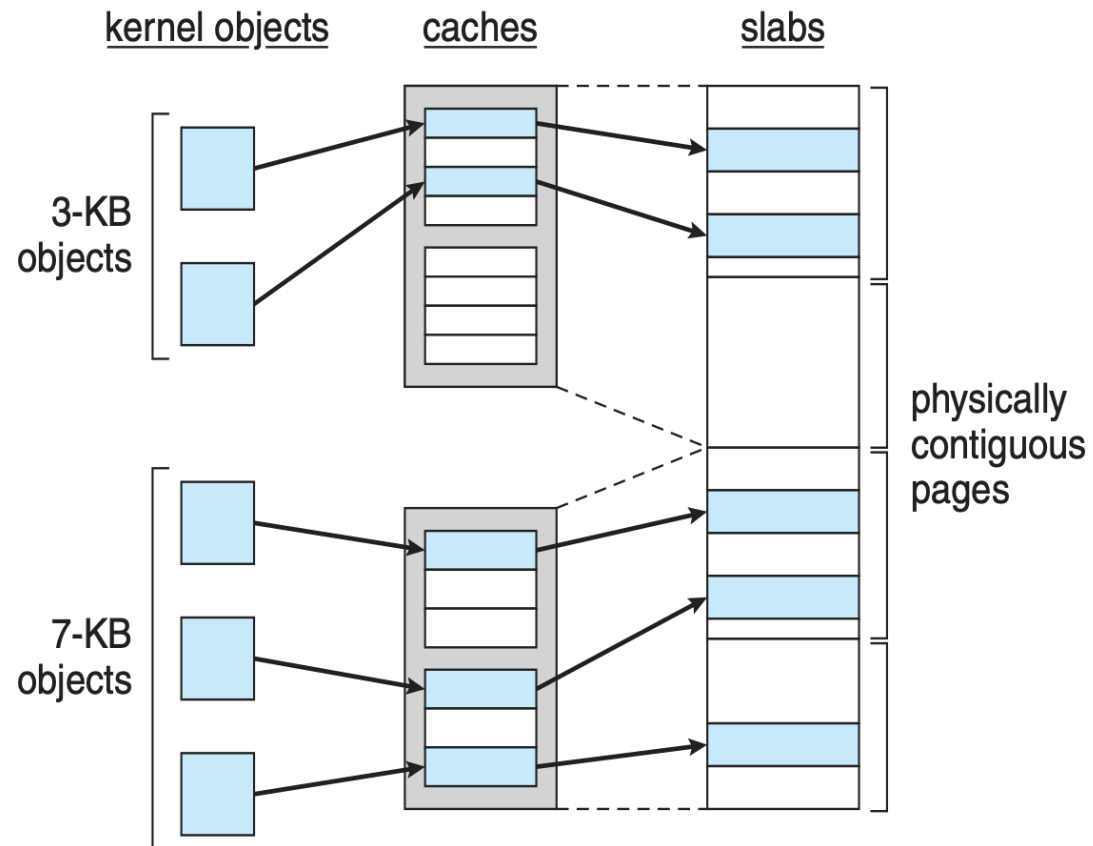
Buddy System Allocator



Slab Allocator

- Slab allocator is a **cache of objects**
 - A **cache** in a slab allocator consists of one or more slabs
 - A slab contains **one or more pages**, divided into **equal-sized objects**
 - Kernel uses one cache for each unique kernel data structure
 - When cache created, allocate a slab, divided the slab into free objects
 - Objects for the data structure is allocated from free objects in the slab
 - If a slab is full of used objects, next object comes from an empty/new slab
- Benefits: **no fragmentation** and fast memory allocation
 - Some of the object fields may be reusable; no need to initialize again

Slab Allocation

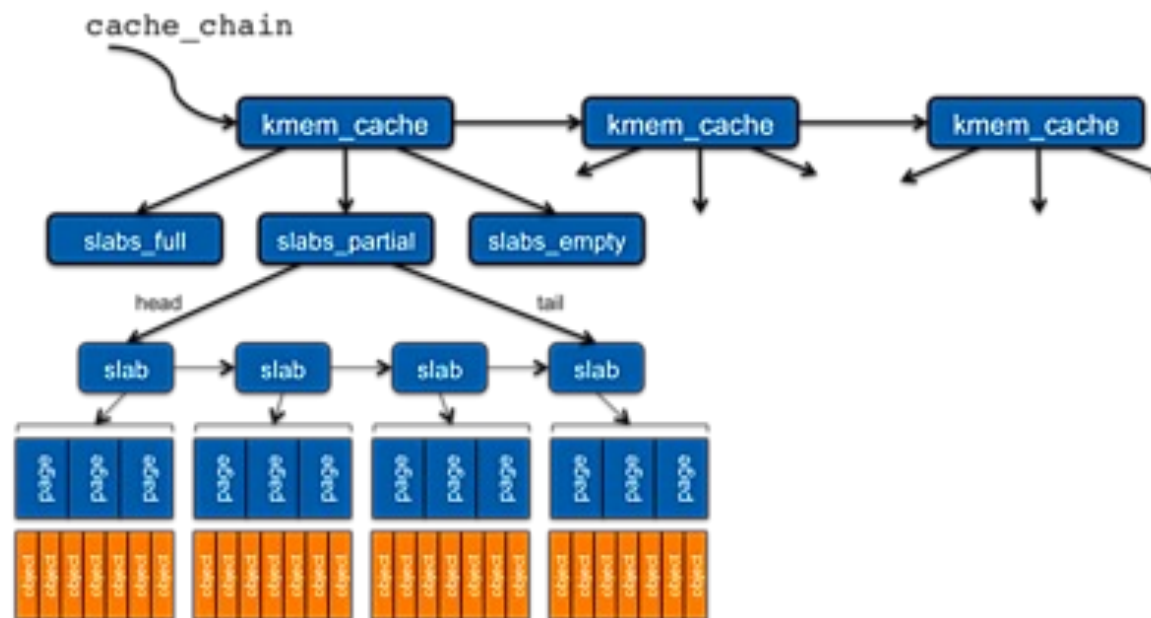


A 12k slab (3 pages) can store 4 3k objects.

Slab Allocator in Linux

- For example process descriptor is of type *struct task_struct*
 - Approx. 1.7KB of memory (some old linux version)
- New task -> allocate new struct from cache
 - Will use existing free *struct task_struct*
- A slab can be in three possible states
 - **Full** – all used
 - **Empty** – all free
 - **Partial** – mix of free and used
- Upon request, slab allocator
 - Uses free struct in **partial** slab
 - If none, takes one from **empty** slab
 - If no empty slab, create new empty

Slab in Linux



Slab Allocator in Linux (Cont.)

- Slab started in Solaris, now wide-spread for both kernel mode and user memory in various OSes
- Linux 2.2 had SLAB, now has both SLOB and SLUB allocators
 - SLOB for systems with limited memory
 - Simple List of Blocks – maintains 3 list objects for small, medium, large objects
- SLUB is performance-optimized SLAB removes per-CPU queues, metadata stored in page structure

Takeaway

- Page fault
 - Valid virtual address, invalid physical address
- Page replacement
 - FIFO, optimal, LRU, 2nd chance
- Thrashing and working set
- Buddy system
- Slab

Memory management summary

- Partition
- Segmentation
- Paging
 - Page table
 - Demand paging
 - Page replacement
 - Working set
- Linux memory mapping
 - 32-bit
 - 64-bit